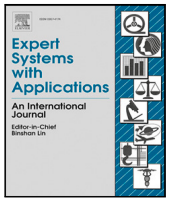
Expert Systems With Applications 239 (2024) 122349



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





Competing leaders grey wolf optimizer and its application for training multi-layer perceptron classifier

Zhenlun Yang

School of Information Engineering, Guangzhou Panyu Polytechnic, Guangzhou, 511483, Guangdong, China

ARTICLE INFO

Dataset link: https://archive.ics.uci.edu/datasets

Keywords:
Multi-layer perceptron
Grey wolf optimizer
Competing leaders
Neural networks
Swarm intelligence algorithm

ABSTRACT

The multi-layer perceptron (MLP) is a highly popular artificial neural network used for classification across various applications. Swarm intelligence algorithms, such as the grey wolf optimizer, battle royale optimizer, sine cosine algorithm, and their improved variants, have been widely utilized for training MLP classifiers, addressing the challenging optimization problem that covers various difficulties in determining the optimal network parameters to minimize errors while processing datasets. Nevertheless, these prevailing algorithms frequently encounter problems like stagnation in local optima and inefficient search, leading to a deficiency in their ability to deliver efficient and accurate solutions. In order to overcome this challenge and provide a powerful tool for MLP classifier training, the competing leaders grey wolf optimizer (CGWO) is proposed in this study. CGWO benefits from a novel mechanism of competing leaders that provides a flexible wolf pack leadership hierarchy to avoid stagnation in local optima and accelerate convergence speed. In addition, a population diversity-enhanced initiation method is designed to help improve the efficiency of the mechanism of competing leaders. The performance of CGWO in training MLP classifiers is evaluated in an experiment comparing CGWO with nine state-of-the-art algorithms on eleven classification datasets representing a wide range of optimization challenges. CGWO substantially outperforms its competitors regarding optimal fitness, classification accuracy, and convergence speed; and shows competitive performance in computation time comparison.

1. Introduction

Heuristics Optimization techniques play a crucial role in addressing complex problems within the realms of science and engineering. Being a popular heuristics optimizer, the Swarm intelligence (SI) algorithm has received significant interest in the research community in recent years. The SI algorithm is mainly based on inspiration from gregarious creatures or natural phenomena, and its typical and well-known examples include Particle Swarm Optimization (PSO) (Li, Li, Wu, You, & Zeng, 2022), Artificial Bee Colony (ABC) (Yavuz, Durmuş, & Aydın, 2022), Cuckoo Search Algorithm (CSA) (Lin, Cao and Zhou, 2022), Grey Wolf Optimizer (GWO) (Mirjalili, Mirjalili, & Lewis, 2014a), Moth–Flame Optimization (MFO) (Yang, 2023), Squirrel Search Algorithm (SSA) (Zhu, Zhou, Sun, & Su, 2022), Honey Bee Mating Algorithm (HBMA) (Jiang, Tsai, Yeh, & Cao, 2017), Firefly Algorithm (FA) (Bacanin et al., 2021), Salp Swarm Algorithm (SSA) (Yang, Shi, Wu, Qiu, & Wei, 2019), Whale Optimization Algorithm (WOA) (Lin, Yu et al., 2022), Bat Algorithm (BA) (Bajaj, Sangwan, & Abraham, 2022), Grasshopper Optimization Algorithm (GOA) (Chakravarthy, Chowdary, Anguera, Mokara, & Satapathy, 2021), Dragonfly Algorithm (DA) (Yang et al., 2022), Battle Royale Optimizer (BRO) (Rahkar Farshi, 2021), Water Cycle Algorithm (WCA) (Taib & Bahreininejad, 2021), and Sine Cosine Algorithm (SCA) (Yang et al., 2023). Due to their characteristics of simplicity, convenience, and robustness, these SI algorithms have been widely admitted in the optimization field and used to solve various optimization problems in the real world (Chakravarthy, Chowdary, Dib, & Anguera, 2022; Jiang et al., 2017). Notably, SI algorithm has also emerged as a valuable asset in enhancing machine learning tools and applications. SI algorithm aids machine learning methods in several aspects, such as feature selection (Bacanin, Budimirovic et al., 2022; Malakar, Ghosh, Bhowmik, Sarkar, & Nasipuri, 2020), model training (Bacanin, Bezdan, Al-Turjman and Rashid, 2022; Yang, 2023), and data clustering (Taib & Bahreininejad, 2021), leading to enhanced accuracy and efficiency. This symbiotic relationship between SI algorithm and machine learning has paved the way for remarkable advancements across various domains, empowering the development of cutting-edge applications ranging from image recognition to natural language processing.

Among the existing optimization problems in machine learning, multi-layer perceptron (MLP) classifiers training is regarded as a challenging task due to it involves different optimization problems with

E-mail address: yangzl@gzpyp.edu.cn.

https://doi.org/10.1016/j.eswa.2023.122349
Received 21 March 2023; Received in revised form 18 October 2023; Accepted 24 October 2023
Available online 29 October 2023
0957-4174/© 2023 Elsevier Ltd. All rights reserved.

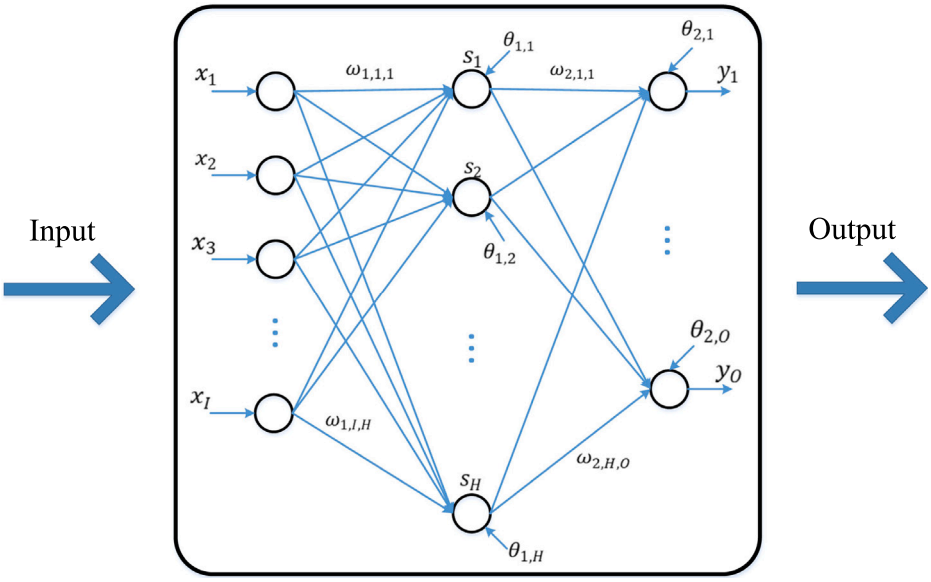


Fig. 1. A three-layer MLP model.

various characteristics. As one of the most popular forms of feedforward neural networks (FNNs), MLP is regarded as a powerful technique in the machine learning field (Tan, Gong, Xu, Li, & Liu, 2023; Yu, Huo, Wang and Li, 2023) and has been employed to solve classification problems in various applications. For example, in the healthcare industry, MLP classifier is useful for predicting the outcome of the existing data, and the prediction assists disease diagnosis and enables early treatment to prevent disease development (Safar, Salih, & Murshid, 2023).

In MLP, neurons are distributed over several fully linked layers, which can be categorized into three levels: the input layer, the hidden layer, and the output layer. Besides the structure, the most crucial part of MLP is a set of parameters, including connection weights and biases, that defines the network relationships of outputs corresponding to inputs and thereby determines the effectiveness of MLP.

Fig. 1 shows a simple model of an MLP classifier with three layers, which contains I input nodes, O output nodes, and H hidden nodes.

After the structure of an MLP classifier are decided, providing the data as inputs and the network parameters, the output of the MLP is calculated through the following equations:

$$s_j = F_1\left(\sum_{i=1}^{I}\left(\omega_{1,i,j}\cdot x_i\right)-\theta_{1,j}\right),\ j=1,2,\ldots,H \tag{1}$$

$$y_j = F_2\left(\sum_{i=1}^{H}\left(\omega_{2,i,j}\cdot s_i\right)-\theta_{2,j}\right),j=1,2,\ldots,O \tag{2}$$

where I indicates the number of inputs, x_i stands for the ith input, $\omega_{1,i,j}$ is the connection weight from the ith input node to the jth hidden node, $\theta_{1,j}$ denotes the bias of the jth hidden node, H is the number of hidden nodes, F_1 is the activation function for the hidden layer, O is the number of nodes in the output layer, F_2 is the activation function for the output layer.

Various forms of activation functions can be employed in MLP. The S-shaped curved *sigmoid* function is one of the most popular activation functions in previous works (Mirjalili, 2015). In this study, the *sigmoid* function is adopted as the activation function for each hidden node (F_1) and the activation function for each output node for binary classification problem (F_2). Whereas, for multi-class classification problems, the activation function for each output node is the $softmax$ function (Bansal, Kumar, Pasrija, & Singh, 2020), which provides the occurrence of the probability to ensure that the sum of all outputs is equal to 1.

Training for the network parameters of an MLP is accomplished according to evolving the weights and biases to minimize the total network error, which can be considered as an optimization problem. The training process of finding the best combination of the network parameters based on the training samples and the corresponding desired outputs significantly affects the performance of the MLP in solving the target problems.

The problem of training MLPs is very challenging and usually has complex characteristics in the search space with many local solutions (Faris, Aljarah, & Mirjalili, 2016). Swarm intelligence algorithms proved efficient in finding a global solution when the search space is challenging. Over past recent years, the well-regarded SI algorithms and their improved variants applied in training MLPs include GWO (Meng, Jiang, & Wang, 2021; Mirjalili, 2015), Battle Royale Optimizer (BRO) (Agahian & Akan, 2022), Sine Cosine Algorithm (SCA) (Gupta, Deep, Mirjalili, & Kim, 2020), Whale Optimization Algorithm (WOA) (Kushwah, Kaushik, & Chugh, 2021), Multi-verse Optimizer (MVO) (Faris et al., 2016), Harris Hawks Optimization (HHO) (Fan, Zhou, & Tang, 2021), Selfish Herds Optimization (SHO) (Zhao et al., 2019), etc.

A well-accepted framework is reported in the literature for training an MLP classifier with swarm intelligence algorithms to optimize the values of the network parameters. Two issues are elaborated in that framework, including the problem variables representation, in other words, the encoding scheme for the search agent and the objective function or fitness function for optimization.

The most commonly adopted encoding scheme is vector encoding which combines the weights and biases into a vector in the order of their appearances from input to output in the neural network. One demonstration of the search agent for the MLP in Fig. 1 is given as follows:

$$vector = \left[\omega_{1,1,1}\cdots\omega_{1,I,H}\ \omega_{2,1,1}\cdots\omega_{2,H,O}\ \theta_{1,1}\cdots\theta_{1,H}\ \theta_{2,1}\cdots\theta_{2,O}\right] \tag{3}$$

After representing the network parameters of MLPs in the form of vectors, an objective function, or a fitness function is required to evaluate each of them. In the framework, the fitness function is defined by the average cross-entropy error (CEE) (Bansal et al., 2020) for all training samples. For calculating CEE, a certain training sample is applied to the MLP, and the error generated by the MLP is calculated using the following equations for binary (Eq. (4)) and multi-class (Eq. (5)) classification, respectively:

$$CEE = -\frac{1}{Z}\sum_{i=1}^{Z}\left(y_i'\log\left(y_i\right)+\left(1-y_i'\right)log\left(1-y_i\right)\right) \tag{4}$$

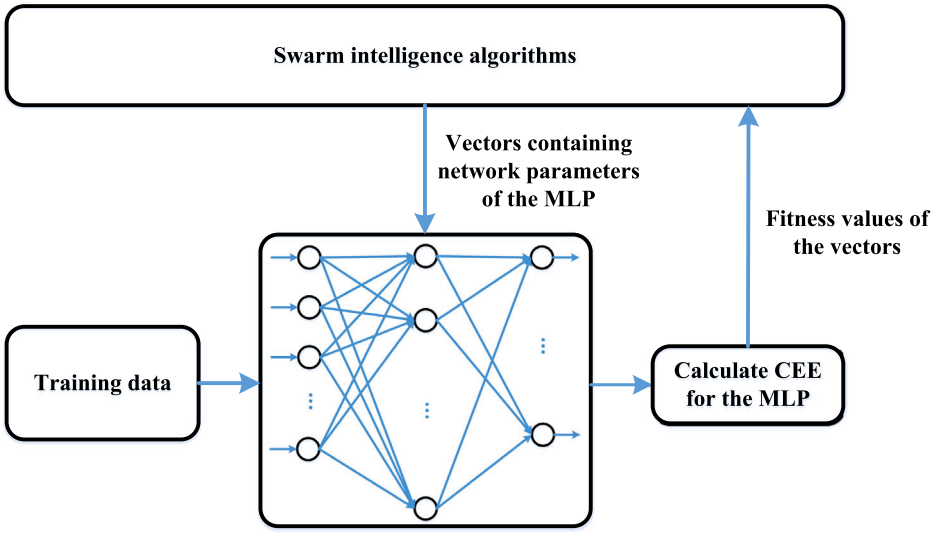


Fig. 2. The MLP training process.

$$CEE = -\frac{1}{Z} \cdot \frac{1}{Q} \sum_{i=1}^{Z} \sum_{j=1}^{Q} \left(y'_{i,j} log \left(y_{i,j} \right) \right) \tag{5}$$

where y'_i and y_i in Eq. (4) are the desired classification result and the actual output of the ith training sample, $y'_{i,j}$ and $y_{i,j}$ in Eq. (5) are the jth desired classification result and the actual output of the ith training sample, Z is the number of training samples, Q is the number of classes.

Based on the search agent representation and the fitness function definition, the MLP training by swarm intelligence algorithms is formulated as a typical optimization problem that needs to minimize the average cross-entropy error generated by the network on all training samples as the inputs. The process of training an MLP using swarm intelligence algorithms is illustrated in Fig. 2. Firstly, the values of the weights and biases of an MLP are set with each search agent of a swarm intelligence algorithm. Then the average cross-entropy error generated by this specified MLP on all training samples is set as the fitness of the corresponding search agent. The swarm intelligence algorithms update the search agents in iterations and finally reach an optimum that has the least average cross-entropy error on all training samples.

Different target datasets under the process by the MLPs always have distinct properties, including the problem dimensions and the shape of the fitness landscape. And thus, they pose different challenges to the optimization algorithm. Although the No-Free-Lunch theorem (Mirjalili, Mirjalili, & Lewis, 2014b) illustrates that no ideal algorithm can provide good solutions for training an MLP for all available datasets, the demand of users for a single tool capable of handling the training tasks of MLPs applied in different datasets with diverse characteristics is reasonable. That is because such desired optimization tools avoid the trial-and-error procedure in selecting a proper algorithm for a specified MLP training. Nevertheless, the existing SI techniques for MLP classifier training have some noticeable drawbacks concerning two issues: (1) Some existing SI-based MLP classifier trainers are designed for specified target applications and cannot be applied to other applications. Such techniques are mostly incapable of meeting the demand for general optimization tools of MLP classifiers. (2) The performance of most existing SI algorithms is insufficient as the general optimization tools for MLP classifiers training, and thus they cannot provide acceptable solutions in many situations. Therefore, to avoid the inefficient trial-and-error method of selecting an algorithm for handling each specified MLP classifier training problem, it is valuable to provide a general efficient optimization tool for MLPs to handle various classification datasets with a wide range of characteristics.

As an emerging swarm intelligence optimization algorithm, the GWO's simulation of the social hierarchy within a wolf pack, along with its distinct movement strategy for search individuals, enables it to efficiently tackle a wide range of complex optimization problems. This capability has been extensively documented in the literature (Emmanuel, Joseph, Oyewola, Fadele, Chiroma, et al., 2021; Faris, Aljarah, Al-Betar, & Mirjalili, 2018; Nadimi-Shahraki, Taghian, Mirjalili, Zamani, & Bahreininejad, 2022). Considering that MLP training optimization involves various optimization difficulties, GWO is regarded as a potential general optimization tool for training MLP classifiers (Mirjalili, 2015). Nevertheless, it is proved in the literature that the search pattern of GWO is weak to provide sufficient global search capability when solving some complex problems, such as some large-scale problems or some multi-modal problems, thus resulting in premature convergence and slow convergence speed (Nadimi-Shahraki, Taghian, & Mirjalili, 2021; Wang, Liu, Li and Khishe, 2022; Wang, Zhu, Ma and Yu, 2022; Zhang, Wang, Chen, Wang, & Fu, 2020).

In this study, to provide an efficient tool for training MLP classifiers for various applications in health care, engineering, agriculture, etc., an enhancement of GWO named competing leaders grey wolf optimizer (CGWO) is proposed. And the CGWO algorithm has three key strengths to improve the efficiency of training MLP classifiers as follows:

(1) A mechanism of competing leaders that features a flexible leadership hierarchy of the wolf pack is proposed to avoid stagnation in local optima and accelerate the convergence speed of the algorithm.

(2) A novel population initialization method based on both chaotic initialization and random initialization enhances the diversity of the population, thus, provides a well-distributed initial population to ensure a more efficient subsequential search.

(3) The CGWO has an acceptable computational efficiency and does not introduce any additional parameters needed to be tuned; thus, it is convenient to be used as the general optimization tool for training MLP classifiers.

The experiment on UCI (University of California) classification datasets that cover the field of healthcare, agriculture, communication, and finance has shown that, in terms of search accuracy, stability, and computational efficiency in training different MLPs with a wide range of problem dimensions, the proposed CGWO algorithm outperforms all the nine competitor algorithms, including the original GWO, five state-of-the-art GWO improved techniques, and three other algorithms reported as the powerful MLPs trainers.

The superior performance of the proposed CGWO algorithm over the state-of-the-art techniques, combined with its advantage of not requiring any parameter configuration, enables researchers to allocate more time and effort to their primary work by employing it as a general optimization tool for training various MLP classifiers, bypassing the

challenges of selecting from a wide range of options of optimization algorithms with various limitations.

In the rest of this paper, GWO and its important improved variants are discussed in Section 2 to provide the research foundation of the proposed CGWO. And the proposed CGWO algorithm is introduced in Section 3. Section 4 provides a comprehensive presentation and analysis of the experimental results, and the significance of the results is proved by statistical analysis. Finally, a conclusion with the future scope of works is given in Section 5.

2. Grey wolf optimizer

2.1. The concepts of GWO

In GWO, the search population consists of two types of grey wolves: wolf leaders (dominant wolves) and follower wolves. In the group of wolf leaders, there are three members: the alpha (α) wolf, representing the best solution found so far in the search space; the beta (β) wolf, representing the second-best solution; and the delta (δ) wolf, representing the third-best solution. The rest of the population are considered as followers, namely omega (ω) wolves. The behavior of the social hunting mechanism of the grey wolf group in GWO includes encircling and hunting. The α, β, and δ are supposed to have better knowledge about the potential location of prey. Therefore, mathematically, the behavior of a grey wolf chasing and encircling the prey is implemented by following the wolf leaders. The corresponding model of each grey wolf in the encircling phase is given in Eqs. (6) and (7) as follows:

$$D_{i,j} = |C \cdot XD_j(t) - X_i(t)| \tag{6}$$

$$U_{i,j}(t) = XD_j(t) - A \cdot D_{i,j} \tag{7}$$

where $X_i(t)$ indicates the ith grey wolf in iteration t, $XD_j(t)$ denotes the jth leading wolf in iteration t, $U_{i,j}(t)$ indicates the target position of the ith grey wolf directed by the jth wolf leader, C and A are coefficient vectors formulated below:

$$C = 2 \cdot r_1 \tag{8}$$

$$A = 2 \cdot a \cdot r_2 - a \tag{9}$$

where r_1 and r_2 are random vectors in [0,1], a indicates a control parameter that linearly decreases from 2 to 0 during the entire search iterations, as expressed by Eq. (10):

$$a = 2 - 2t/T \tag{10}$$

where t and T are the current iteration number and the maximum iteration number, respectively.

Based on the model of the encircling phase, the movement of each grey wolf in each iteration is determined in the hunting phase. Since the wolf leaders group contains three members, the movement of each grey wolf consists of three components accordingly. The following equation is adopted for the hunting phase of each grey wolf in this regard:

$$X_i(t+1) = \sum_{j=1}^{3} U_{i,j}(t)/3 \tag{11}$$

The position of each grey wolf is determined by the average of the target positions directed by the three wolf leaders. After each search iteration, a wolf leader will be replaced by an ω wolf if the ω wolf reaches a better position than the wolf leader. The pseudocode for the GWO algorithm is given in Algorithm 1, in which x_{lb} and x_{ub} are the vectors representing the lower and upper bounds of the search space. Specifically, each element of x_{lb} denotes the lower bound of the corresponding dimension in the search space, while each element of x_{ub} denotes the upper bound of the corresponding dimension in the search space. N and D refer to the search population size and the problem dimension, respectively. Further details can be found in the literature (Mirjalili et al., 2014a).

Algorithm 1 The pseudocode of GWO

```
for i=1 to N do
    Generate a D-dimensional vector R, with elements within [0,1] randomly.
    X_i = x_lb+R · (x_ub - x_lb);
    calculate the objective function value of X_i;
end for
set the best three wolves as the wolf leaders XD_α, XD_β, and XD_δ;
while the maximum number of objective function evaluations (FEs) is not reached do
    update a with Eq. (10);
    for i=1 to N do
        update the position of X_i with Eqs. (6), (7), (8), (9), (11);
    end for
    for i=1 to N do
        calculate the objective function value of X_i;
    end for
    for i=1 to N do
        if X_i has a better objective function value than XD_α then
            replace XD_α with X_i;
        else if X_i has a better objective function value than XD_β then
            replace XD_β with X_i;
        else if X_i has a better objective function value than XD_δ then
            replace XD_δ with X_i;
        end if
    end for
end while
```

2.2. GWO improved variants

Due to its robustness and easy implementation, GWO has been applied to various application scenarios, such as mechanical engineering design (Banaie-Dezfouli, Nadimi-Shahraki, & Beheshti, 2021), medical diagnosis (Li et al., 2017), power system control (Guha, Roy, & Banerjee, 2016), and photovoltaic solar cell model estimate (AlShabi, Ghenai, Bettayeb, Ahmad, & El Haj Assad, 2021). However, when solving some complex optimization problems, such as large-scale problems or multi-modal problems, GWO lacks sufficient global convergence ability and suffers from poor search efficiency, which results in local optima trapping and slow convergence speed (Nadimi-Shahraki et al., 2021; Wang, Zhu et al., 2022). Hence, to overcome these weaknesses, various improved versions of GWO have been proposed in recent years. The main approaches focus on improving GWO from three different aspects, including (1) changing the behavior of the grey wolves for hunting, (2) changing the social leadership hierarchy of the grey wolf group, and (3) integrating the search patterns from other algorithms into GWO.

In the first category approaches, the behavior of the grey wolf group is modified by adding or removing some terms of the position update equations or by changing the generating method of the corresponding terms. For example, to improve the exploration ability, the grey wolf is updated with one of the five best solutions as the search center in the Representative-based GWO (R-GWO) (Banaie-Dezfouli et al., 2021). Instead of following the original linear decrease scheme, the exploration/exploitation parameter, a, is adaptively tuned according to the significance of the change of the search agents' average fitness improvement in the Adaptive GWO (AGWO) (Meidani, Hemmasian, Mirjalili, & Barati Farimani, 2022). Moreover, the stop mechanism of AGWO is also based on the significance of fitness improvement in iteration; the evolution process is stopped when the fitness improvements become negligible. In Mutation-Driven Modified GWO (MDM-GWO) (Singh & Bansal, 2022), when determining the final position of each grey wolf, the three components obtained in the direction of each wolf leader

($U_{i,j}(t)$ in Eq. (7)) are not assigned equal weights as stated in Eq. (11). Instead, the three $U_{i,j}(t)$ are multiplied by random weights respectively, and the three results are summed up to generate the $X_i(t+1)$, thereby the wolf's ability to escape from local optima is enhanced. Similarly, in Reinforced Exploitation and Exploration GWO (REEGWO) (Yu, Xu, Wu, & Wang, 2022) and Advanced GWO (AGWO) (Meng et al., 2021), different weights with inverse proportions of the wolf leaders' fitness values are assigned to the components generated by Eq. (7) to perform the reinforcement exploitation. In RSMGWO (Zhang et al., 2020), other than the original search strategy of GWO, each grey wolf uses some different updating modes, including random opposition learning, differential mutation, and Gaussian mutation, to improve the optimization performance. In Ensemble GWO (EGWO) (Yu & Wu, 2022), a dedicated search strategy is designed solely for the three wolf leaders, enabling them to conduct searches around themselves based on their relative distances from the other wolf leaders, thereby enhancing the exploitation search.

The improved versions of GWO in the second category focus on improving the search efficiency of GWO by changing the social leadership hierarchy of the wolf pack, in other words, the roles of the grey wolves for hunting. In Incremental GWO (IGWO) (Seyyedabbasi & Kiani, 2021) and Expanded GWO (ExGWO) (Seyyedabbasi & Kiani, 2021), by using the position update equations of GWO, each grey wolf is updated with the wolf leader(s) defined in the original GWO and the wolves previously updated in the same iteration. In other words, for a certain wolf, besides α, β and δ, the wolves previously updated in the same iteration are also regarded as the wolf leaders. Therefore, the number of wolf leaders for each grey wolf is different. The main difference between IGWO and ExGWO is that the minimum number of wolf leader(s) involved in the position update for each search agent is 1 for IGWO and 3 for ExGWO. In Group-based Synchronous-Asynchronous GWO (SA-GWO) (Rodríguez et al., 2021), to achieve a better balance between exploration and exploitation, the population is divided into three subgroups and each subgroup is individually processed with a certain probability of interchange of elements among the subgroups. Similarly, to achieve better robustness and faster convergence speed, the Multi-Group GWO (MG-GWO) (AlShabi et al., 2021) divides the grey wolves into multiple subgroups and updates each grey wolf with the wolf leaders in each subgroup. In sub-population improved GWO with Gaussian mutation and Levy flight (SPGWO) (Yu, Duan and Cai, 2023), the population is equally divided into superior and inferior two sub-populations by fitness value and updated with different strategies based on the fitness values comparison between the two sub-populations.

The GWO variants in the third category can be classified as hybrid algorithms. Two novel search strategies are designed by combining the position update equations of GWO and PSO and used to update the search agents alternatively in Hybrid Particle Swarm and GWO (HGWOP) (Zhang et al., 2021). In Sine Cosine GWO (SC-GWO) (Gupta, Deep, Moayedi, Foong, & Assad, 2021), a modified sine cosine algorithm and GWO are run in sequence to establish a comparatively better synergy between the exploration and exploitation of the algorithm. In PSOGWO (Adnan et al., 2021), to improve the capability of PSO, which acts as the main optimization algorithm, of escaping from local optima, the best solutions generated by the assistance algorithm, GWO, are used to substitute some particles of PSO. For balancing the local and global search and maintaining diversity, the Improved GWO (IGWO) (Nadimi-Shahraki et al., 2021) introduces a differential approach, namely dimension learning-based hunting (DLH), to construct a neighborhood for each wolf in which the neighboring information can be shared between wolves. Subsequently, the IGWO determines the new position of each search agent by selecting the superior candidate from the two candidates generated through the GWO and DLH search strategies. In other words, the candidate with a better fitness value is chosen as the search agent's new position. In local opposition-learning golden-sine GWO (OGGWO) (Li, 2023), once the standard position update procedure of GWO is completed, each grey wolf undergoes an additional update using the golden sine spiral motion method. This supplementary step is employed to enhance the algorithm's global search ability and accelerate its convergence speed.

3. The proposed CGWO algorithm

For processing complex optimization problems like MLP training, the crucial characteristic of the metaheuristic algorithm is the capability of exploring unknown areas of the search space to escape the local optima and exploiting the promising areas confirmed so far to approximate the optimal results at a reasonable rate.

In the proposed CGWO algorithm, the mechanism of competing leaders is proposed and used to alleviate the problem of insufficiency in global and local search abilities and thus improve the ability to approximate the global optimum and accelerate the convergence speed. Moreover, a new population initiation method is also designed to improve the initial population diversity, ensuring a more efficient subsequential search of the algorithm. Further, CGWO inherits the superiority of the easy implementation and low computational complexity from GWO, does not require tuning any additional parameters and has an acceptable computational efficiency for practical applications.

In what follows, the motivation and the implementation of the mechanism of competing leaders are presented first, then the procedure of CGWO will be introduced in the second subsection, and the computational complexity of CGWO will be discussed in the third subsection.

3.1. The mechanism of competing leaders

3.1.1. Motivation

In GWO, the strategy of multiple search centers brought by the three leaders is designed to achieve exploration for each search agent attracted by the three leaders from different directions assisted by the random terms in the position update equations. The wolf in GWO is obliged to follow the leaders and search in different areas discovered so far with the potential to find the prey (optimum). The hunting process is terminated when the wolf leaders confirm the position of the prey that has stopped moving. The exploration and exploitation are controlled mathematically with the value of a in Eq. (10). The search behaviors of the wolves in the first half of the iterations focus on exploration, whereas the other half of the iterations are dedicated to the exploitation (Emary, Zawbaa, & Grosan, 2017).

The exploration capability of SI algorithms allows them to systematically venture into unexplored regions of the search space, uncovering potentially more optimal or promising solutions than those already explored. In the optimization problem of MLP training, it is stated in the literature that exploration is crucial for approximating a satisfactory result (Mirjalili, 2015). More measures, such as having more random and abrupt search steps, should be taken to enhance the diversity of the population when processing complex datasets with MLPs (Mirjalili et al., 2014b). However, the search pattern employed by GWO may potentially undermine the effectiveness of exploration, even during the initial half of iterations. If each of the three wolf leaders is close to one another, the multiple search centers practically tend to degenerate into a singular search center. The scheme of a single search center reduces opportunities for search individuals to learn from other excellent solutions, resulting in a rapid loss of population diversity of swarm intelligence algorithms, as reported in the literature (Wang, Hao, Chen, Wang, & Jiang, 2021). Despite the utilization of a relatively large value for parameter a at this stage, which promotes extensive exploration around the search center as indicated by Eqs. (7), (9), and (10), the population's diversity diminishes rapidly due to the reliance on a singular search center in the search pattern. Especially when the three wolf leaders get stuck in a local optimum simultaneously, inefficient search is inevitable, and the search agents are less likely to escape from the local

optimum. As an example, under the experimental analysis, it is founded that the three-leaders scheme reduces the diversity of the population and causes GWO to fall into the local optimum and, finally, premature convergence (Nadimi-Shahraki et al., 2021). On the other hand, in each iteration, the three wolf leaders of GWO are not updated until all search agents are repositioned and evaluated, and this strategy weakens the exploitation search around promising areas and results in slow convergence speed. In a research case, the slow convergence speed of GWO is demonstrated in an experiment (Wang, Zhu et al., 2022). Due to the two aspects of drawbacks in the search pattern, when solving complex optimization problems such as large-scale problems (Zhang et al., 2020) or multi-modal problems (Wang, Liu et al., 2022), GWO lacks the power to reach a globally optimal solution or has a slow convergence speed, or even suffers from both aspects of poor performance. For example, the experiment carried on the well-known nineteen benchmark functions with one hundred dimensions illustrates the poor performance of GWO in large-scale optimization (Zhang et al., 2020). Regarding the multi-modal problems optimization, in the experiment conducted on the test set consisting of fifteen widely used multimodal numerical test functions, the performance of GWO is unsatisfactory (Wang, Liu et al., 2022).

To tackle this problem, a mechanism of competing leaders for the wolf group is proposed and improves the GWO in two manners: (1) it introduces the wolf leaders with an extensive membership in the initial stage, whereas the number of leaders decreases over the course of competition; (2) it provides a compact strategy for identifying the newly found good solutions as the wolf leaders.

3.1.2. Extensive leaders and their competition

In the first technique of the competing leaders mechanism, to improve population diversity and thus enhance exploration search, the initial number of wolf leaders is set to a value greater than the default number of wolf leaders in GWO to ensure that each search agent receives diverse guidance for their search process. As the evolutionary iteration progresses, the focus of the algorithm gradually shifts from exploration to exploitation. This shift is accompanied by a decrease in the number of wolf leaders, and those wolf leaders with better fitness values are retained while those with poorer fitness values are dismissed.

Thus, over the course of iterations, the number of the wolf leaders, M, is mathematically defined as follows:

$$M = \begin{cases} \left[M_{max} - t * \frac{4*(M_{max}-M_{min})}{3*T} \right], & if\ t < \frac{3}{4*T} \\ M_{min}, & if\ t \geq \frac{3}{4*T} \end{cases} \tag{12}$$

where M_{max} and M_{min} are the maximum and the minimum values of M, respectively, and [ ] is the rounding operator.

It can be seen that M_{max} is the initial number of the wolf leaders whereas M_{min} is the final number of the wolf leaders, the number of wolf leaders linearly decreases from M_{max} to M_{min} in the first phase, and after, it keeps constant as M_{min} for the rest of iterations.

This scheme simulates the competing process of the wolf leaders. Initially, a relatively large number of leaders affect the search behavior of the population. And as the iteration progresses, some former leaders lose their dominating roles in the competition with other leaders and become members of followers. Finally, only a small number of leaders remain and guide the population to converge to the final optimum.

For the mechanism of competing leaders, the initial and the final values of the number of wolf leaders, M_{max} and M_{min}, have an important influence on the performance of the algorithm. The value of M_{min} could be set to three. It means that, in the later iterations, the scheme of three leaders from the original GWO is adopted to focus on the exploitation search. But the issue of setting M_{max} is more complicated. Even in the exploration stage, search efficiency is one of the key issues that must be concerned. If M_{max} is set at an improperly enormous value, it leads to the problem of no proper and definite search centers for the population; thus, it results in the lack of search efficiency in the early iterations. On the other hand, if the value of M_{max} is too tiny, then the effect of the mechanism of competing leaders is weakened, causing the algorithm to degenerate into the original GWO. To tackle this issue, a scheme of setting M_{max} according to the population size and the target problem dimension is proposed. Obviously, a large-scale optimization problem implies a more extensive search space, and to effectively explore these search spaces, a more diversified population is essential. Therefore, for larger-scale problems, it is crucial to establish a larger group of wolf leaders capable of maintaining the diversity of the population, whereas, for small-scale and medium-scale problems, a relatively small number of wolf leaders is sufficient. Taking this into consideration, the equation of setting M_{max} is given as follows:

$$M_{max} = \begin{cases} \left[\frac{D}{N} * \frac{MX_u - MX_l}{4} + MX_l \right], & if\ \ D \leq 4 * N \\ MX_u, & if\ \ D > 4 * N \end{cases} \tag{13}$$

where N is the population size, D is the problem dimension, MX_u and MX_l are the upper and lower bounds of M_{max}. From Eq. (13), it can be seen that when solving small-scale and medium-scale problems, M_{max} is set at a relatively small value. And for solving larger-scale problems, M_{max} is set at its upper bound value.

In practice, the value of MX_u can be set to a constant value that is a multiple of M_{min}. And the value of MX_l can be set to the same value as M_{min}. The recommended values for MX_u and MX_l are given as follows:

$$MX_u = \left(M_{min} + 1 \right) * 10 \tag{14}$$

$$MX_l = M_{min} \tag{15}$$

Following the mechanism of competing leaders, The final step of the position update equation for each wolf is modified as follows:

$$X_i(t+1) = \sum_{j=1}^{M} U_{i,j}(t)/M \tag{16}$$

where $U_{i,j}(t)$ is calculated with Eq. (7).

By examining Eqs. (6) and (7), It can be observed that each $U_{i,j}(t)$ is determined by the current position of a search agent and the position of a particular wolf leader. Each $U_{i,j}(t)$ can thus be regarded as a target position of the search agent directed individually by the specified wolf leader. Therefore, the calculation of the final position of each search agent using Eq. (16) can be considered as the mean filtering on the set containing the independent influence of each wolf leader on the current search agent.

Figs. 3 and 4 show how the mean filtering effect work on the position update of the search agents and illustrate the mechanism of competing leaders with more details from another analytical perspective. From this point of view, the main difference between Eqs. (11) and (16) is the different window sizes for the mean filter, which is demonstrated in Fig. 3(A) and (B), respectively. The mean filter is extensively applied to denoise images (Guo, Wang, Han, Song, & Yang, 2022), and it is capable of smoothing the influence of a certain wolf leader on the search agents. A higher level of window sizes has a more significant degree of blurring effect on the influence of each wolf leader on the search agents. Therefore, as demonstrated in Fig. 4, in the proposed competing leaders approach, the window size is set to a relatively large value to encourage the population to search with more diverse behaviors in the earlier stage. And in the later stage, a small window size is used to guarantee the convergence of the population towards current top leaders. The implementation of the mean filter on the wolf leaders can be described by another form of Eq. (16), specifically Eq. (17), as follows:

$$X_{i,k} = \langle \left[U_{i,1,k} \ \cdots \ U_{i,M,k} \right], \left[1/M \ \cdots \ 1/M \right] \rangle, k = 1, 2, \ldots, D \tag{17}$$

where $X_{i,k}$ is the kth dimension of the ith wolf after its position update, D is the dimension of the search space, $\langle\rangle$ returns the inner product of the two vectors.

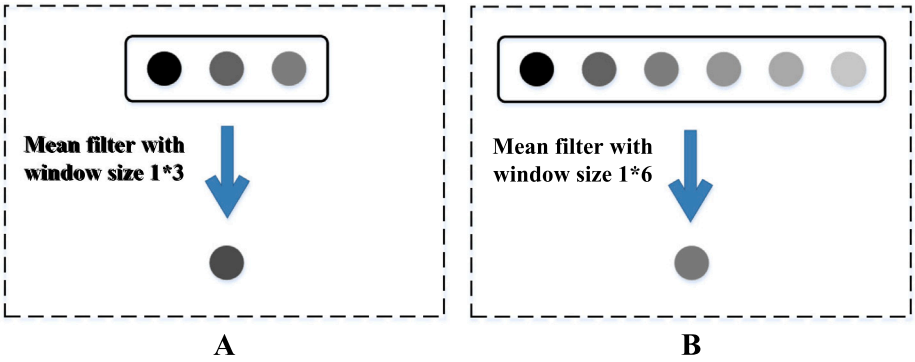


Fig. 3. The mean filtering effect of Eqs. (11) and (16).

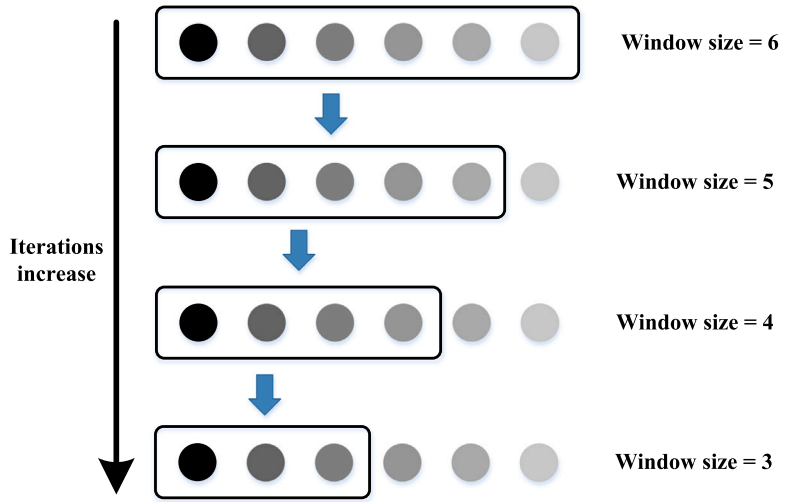


Fig. 4. The decreasing window size for wolf leaders in the competing leaders approach.

3.1.3. The compact wolf leaders update strategy

The second technique of the mechanism of competing leaders, namely the compact wolf leaders update strategy (CWLU), provides a greedy scheme to guarantee the leader wolves are the best solutions found so far when the position of each wolf is updated.

According to the mechanism of hunting behaviors of the wolves in GWO, wolf leaders can be regarded as pins or flags marked by members of the wolf population in searching for the optima. Although the diversity of the population is enhanced by the mechanism of competing leaders, the large leader group scheme smooths the influence of every single leader acting on the population. As a result, the hunting behaviors of the wolves may not be sensitive enough when a new promise area is found and marked by one of the leaders. And if some members of the leaders are not good enough, an inefficient search is inevitable.

The CWLU strategy can shorten the waiting period required for better leaders to influence the search behaviors of the wolves, and it is beneficial for guiding the wolves to perform the search more efficiently. Therefore, the CWLU strategy is an effective complementary method for the scheme of extensive leaders, helping maintaining population diversity without weakening the high-efficiency search of the population. The two strategies of the mechanism of competing leaders cooperate to achieve the balance between exploitation and exploration of the algorithm, in which the well-distributed population is emphasized in earlier iterations, and the population gradually tilts towards convergence and later achieves rapid convergence when the evolution progresses.

To demonstrate the main idea and the benefit of the CWLU strategy, the comparison of the canonical wolf leaders update approach in GWO and the CWLU approach is illustrated in Fig. 5. Fig. 5(A) refers to the canonical wolf leaders update approach, and Fig. 5(B) refers to the CWLU approach. The positions of the wolves are updated sequentially, as in t_1, the position of wolf 1 is updated, subsequently, the position of wolf 2 is updated in t_2, and so on. The circles in black or grey are the current wolf leaders, and the darker, the better on fitness value. The circle in red indicates a better position than each of the circles in black or grey, while the circle in blue indicates a worse position than each of the circles in black or grey. In the canonical approach of wolf leaders update in GWO, when a wolf reaches a better position, the position would not be regarded as one of the leaders' positions until all the wolves are updated in the same iteration. Therefore, even if a wolf reaches a better position, the subsequent position update for the rest of the wolves in the same iteration cannot benefit from it, as illustrated in Fig. 5 (A). Whereas, in the CWLU approach, the better position is used to update the set of wolf leaders right after it is found. Therefore, the subsequent wolf is more likely to be guided to a more promising area; in other words, it benefits from the newly found information immediately, as illustrated in Fig. 5 (B).

In addition, the CWLU approach supports a more meticulous search. The wolf leaders are gradually updated with the newly found positions covering wider areas along the less optimal points to the more optimal points. In this way, the search step of the wolves was set to a more reasonable value, and the oversized step that leads to low search accuracy is more likely to be avoided. In contrast, in the canonical approach of wolf leaders update, the newly found positions better than the older leader wolves but worse than the final several best positions in the current iteration were ignored.

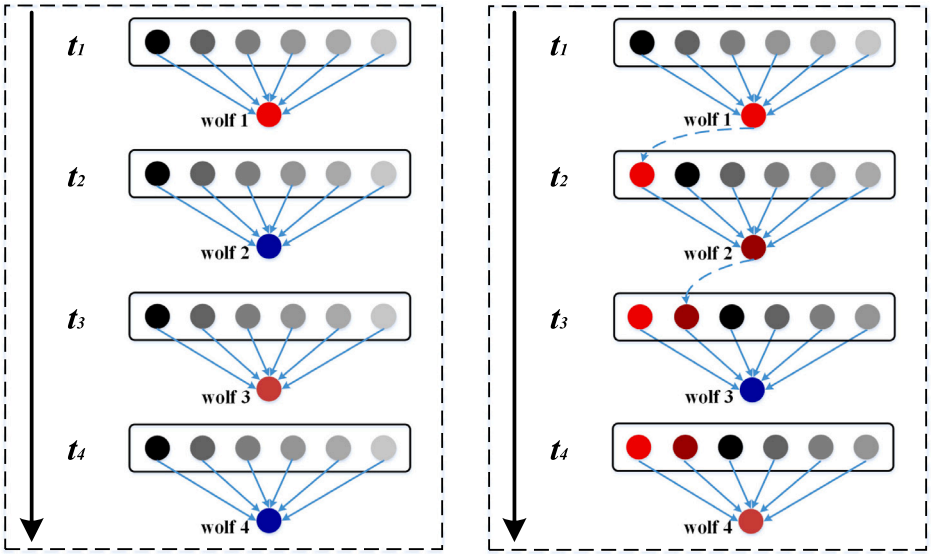


Fig. 5. The principle of the CWLU strategy.

3.2. Procedure of CGWO

The main differences between GWO and CGWO lie in three aspects: (1) the population initialization method (for CGWO, an approach namely initiation with population diversity enhancement, IPDE for short, is employed), (2) the position update equations, and (3) the timing of updating the members of wolf leaders. The pseudocode for CGWO is presented in Algorithm 2, and these three aspects will be described one by one in the following content.

Algorithm 2 The pseudocode of CGWO

```
initiate the population, with a size of N, using the IPDE approach;
set M to the value of M_max;
set the best M wolves as the wolf leaders XD_1, XD_2, ..., XD_M;
while the maximum FEs is not reached do
    update a with Eq. (10);
    update M with Eq. (12);
    for i=1 to N do
        update the position of X_i with Eqs. (6), (7), (8), (9), (16);
        calculate the objective function value of X_i;
        if X_i is better than one of the wolf leaders then
            replace the worst one of the wolf leaders with X_i;
        end if
    end for
end while
```

3.2.1. Population initialization method

As reported in the literature (Gao & Liu, 2012), the diversity of the initial population in distribution is crucial for the subsequent optimization process of the algorithm by affecting the convergence speed and the quality of the final reached optimum. An initial population distributed diversely along the search space is beneficial for the algorithm to explore more promising regions and eventually discover optimal solutions during the optimization process. On the other hand, a population distributed narrowly is more likely to get trapped in local optima, resulting in poor optimization results. Random initialization is the most commonly adopted method when no prior information about search space is available. However, purely random resampling solutions within the search space boundaries have the chance of visiting or even revisiting unproductive regions of the search space, which may lead to a lack of diversity in the population. Therefore, in order to improve the efficiency of the mechanism of competing leaders, the IPDE population initiation approach is employed in CGWO.

In the IPDE approach, two population are initialized with two different methods. Since the nature of chaos is random, unpredictable and sensitive, dependent upon its initial condition, it is beneficial for extracting the search space information to increase the population diversity (Teng, Lv, & Guo, 2019). And according to the literature (Mirjalili & Gandomi, 2017), the sinusoidal chaotic map has exhibited good performance in assisting the algorithms to explore unknown areas. Therefore, in the proposed approach, the first method employs the sinusoidal chaotic map to initialize the population. The sinusoidal iterator is defined as follows:

$$r_k = \sin\left(\pi * r_{k-1}\right), r_{k-1} \in (0,1),\ k = 1,2,\ldots,K \tag{18}$$

The initialization based on the mapped variables r_k in Eq. (18) can generate the individuals distribute in search space with ergodicity, randomness, and irregularity.

Moreover, in order to improve the applicability of the initialization approach in solving problems of different dimensional scales, the traditional random initialization method is also employed to generate the second population. In our implementation, the random initialization uses the random values obeying the Gaussian distribution to directly generate each individual within the boundaries of the search space. Once the two populations are generated and evaluated using the objective function (the CEE is calculated using Eqs. (4) or (5) in this study), a new population for subsequent iterations is formed by selecting the individuals with fitness values ranking in the top half from the union of the two populations.

The pseudocode for the IPDE approach is presented in Algorithm 3, in which x_{lb} and x_{ub} have the same meaning as that in Algorithm 1, representing the lower and upper bounds of the search space. It should be noticed that each dimension of the vector of a search agent represents a distinct member of the set of network parameters of an MLP. Moreover, the lower and upper bounds of each dimension of the search space are determined by the defined ranges of the corresponding network parameters.

3.2.2. The position update equations

According to the mechanism of competing leaders, one of the position update equations of CGWO is different from GWO; that is, in CGWO, Eq. (11) is substituted by Eq. (16). And to cooperate with the mechanism of competing leaders and determine how many of the wolf

Algorithm 3 The pseudocode of IPDE approach

$IX1 = \emptyset$; $IX2 = \emptyset$;
for i=1 to N **do**
  Generate a D-dimensional vector R, with elements within [0,1] randomly.
  $ix1_i = x_{lb} + R \cdot (x_{ub} - x_{lb})$;
  calculate the objective function value of $ix1_i$;
  $IX1 = IX1 \cup \{ix1_i\}$;
end for
for i=1 to N **do**
  Generate a D-dimensional vector R_0, with elements within [0,1] randomly.
  for k=1 to K **do**
    $R_k = \sin(\pi \cdot R_{k-1})$
  end for
  $ix2_i = x_{lb} + R_K \cdot (x_{ub} - x_{lb})$;
  calculate the objective function value of $ix2_i$;
  $IX2 = IX2 \cup \{ix2_i\}$;
end for
Select N fittest individuals from $IX1 \cup IX2$.

leaders are involved in guiding the population, the value of M must be calculated in each iteration according to Eq. (12). If the new value of M is less than its former value, the size of the leader group will be shrunk by removing the members with the worst fitness values.

3.2.3. The timing of updating the members of wolf leaders

Compared with the original GWO, the CWLU strategy causes a different timing of updating the members of wolf leaders. The procedure of the CWLU strategy is straightforward, and when each wolf is updated and evaluated by the fitness function, it will compare with the members of the wolf leaders. If the newly updated wolf has a better fitness value than the least superior member of the wolf leaders, it will substitute the corresponding leader wolf.

3.3. Time complexity analysis

The time complexity refers to the run time required by an algorithm when it is used to solve a problem by the corresponding executable program. It is a crucial metric capable of evaluating an algorithm's performance on efficiency in another aspect. The worst time complexity of the proposed CGWO algorithm is calculated and compared with the original GWO in terms of big-O notation by analyzing the pseudocodes presented in Algorithms 2 and 1.

The computational complexity of the CGWO and GWO is determined by five subsequent steps: population initiation, parameters updating, wolf population position updating, objective function evaluations, and the wolf leaders updating. The time complexity of an algorithm relays on the population size N and the maximum number of iterations T. The time complexity analysis on GWO and CGWO is given as follows:

(1) The time complexity of the population initiation in lines 1 to 5 of Algorithm 1 is $O(N)$ for GWO, and the time complexity of the population initiation in line 1 of Algorithm 2 is $O(N)$ for CGWO.

(2) The time complexity of the parameters updating operation in line 8 of Algorithm 1 is $O(T)$ for GWO, and the time complexity of the parameters updating operation in lines 5 and 6 of Algorithm 2 is $O(T)$ for CGWO.

(3) The time complexity of the wolf population position updating operation in line 10 of Algorithm 1 is $O(TN)$ for GWO, and the time complexity of the wolf population position updating operation in line 8 of Algorithm 2 is $O(TN)$ for CGWO.

(4) The time complexity of the objective function evaluations in line 13 of Algorithm 1 is $O(TN)$ for GWO, and the time complexity of the objective function evaluations in line 9 of Algorithm 2 is $O(TN)$ for CGWO.

(5) The time complexity of the wolf leaders updating operation in lines 15–23 of Algorithm 1 is $O(TN)$ for GWO, and the time complexity of the wolf population position updating operation in lines 10–12 of Algorithm 2 is $O(TN)$ for CGWO.

According to the analysis above, the total time complexity of GWO and CGWO is the same and which is $O(TN)$.

Even though CGWO requires more computation time than GWO due to the presence of more operations in CGWO, it can be inferred that the additional computation time of CGWO is not more than a fixed proportion of the computation time of GWO since the time complexity of CGWO is the same as that of GWO in terms of big-O notation.

Table 1
Datasets for experiment.

No.	Datasets	No. of attributes	No. of classes	No. of samples	MLP structure	Problem dimension
1	Scale	4	3	625	4-9-3	75
2	Seeds	7	3	210	7-15-3	168
3	Breast cancer	9	2	699	9-19-1	210
4	Statlog	13	2	270	13-27-1	406
5	Wine	13	3	178	13-27-3	462
6	Australian	14	2	690	14-29-1	465
7	Cleveland	13	5	303	13-27-5	518
8	Zoo	16	7	101	16-33-7	799
9	Parkinson	22	2	195	22-45-1	1081
10	Ionosphere	34	2	351	34-69-1	2485
11	Chess	36	2	3196	36-73-1	2775

4. Experimental study

In this section, to demonstrate the performance of the proposed CGWO algorithm in MLP classifier training, an experimental simulation was conducted on the UCI datasets to search for the connection weights and biases of the MLPs. Firstly, the experimental setup is presented, including details on the used datasets, the compared algorithms, the methodology, and the parameter settings, providing a comprehensive overview of how the experiment was conducted. Following that, the experimental results and discussion are provided.

4.1. Experimental setup

4.1.1. Datasets

To conduct a reliable comparison, eleven well-known benchmark datasets with varying numbers of features, outputs, and instances were selected from the UCI Machine Learning Repository (Aha et al., 2021) to evaluate the performances of the competing algorithms in training MLP classifiers.

Table 1 provides an overview of these datasets, including details such as the number of attributes for each data sample, the number of distinct classes included in the database, the total number of samples, the corresponding MLP structure, and the dimensions of the problem. It should be noticed that the dimension is calculated based on the length of the vector in Eq. (3).

Firstly, considering the representativeness of databases in practical applications of MLP classifiers, five of the datasets are related to the healthcare industry, which is an important application field of MLP classifiers, and the rest datasets cover agriculture, communication, and finance. Secondly, to represent a wide range of optimization challenges to the competing algorithms in training MLPs, the datasets with various features were selected for the experiment. These datasets impose various difficulties on the competing algorithms to ensure that the capabilities of the competing algorithms to address problems associated with MLP training are comprehensively evaluated. Many researchers have adopted part of these datasets to evaluate their proposals (Agahian

& Akan, 2022; Mirjalili, 2015). Among the eleven datasets, six contain two classes, three contain three classes, and two contains more than three classes. The numbers of features of the datasets are from 4 to 36.

According to the literature, there is no standard rule for selecting the number of neurons in the hidden layer. In this work, we follow the method suggested in the literature (Mirjalili, 2015), and the number of hidden nodes equals to $2 * I + 1$, where I is the number of features in the dataset. Then the corresponding numbers of decision variables of the datasets are from 75 to 2775. A large-scale optimization problem refers to a problem with over 100 variable dimensions (Mahdavi, Shiri, & Rahnamayan, 2015), and thus, ten of the datasets in the experiment yield large-scale optimization problems.

4.1.2. The compared algorithms

The original GWO (Mirjalili, 2015), five improved versions of GWO, including IGWO(M) (Nadimi-Shahraki et al., 2021), AGWO (Meng et al., 2021), RSMGWO (Zhang et al., 2020), ExGWO (Seyyedabbasi & Kiani, 2021), and IGWO(A) (Seyyedabbasi & Kiani, 2021), together with three other remarkable optimization algorithms, including BRO (battle royale optimizer) (Agahian & Akan, 2022), MSCA(modified sine cosine algorithm) (Gupta et al., 2020), and RDE-OP (region-based differential evolution algorithm incorporation opposition-based learning) (Mousavirad, Schaefer, Korovin, & Oliva, 2021) were adopted as the algorithms for comparison with the proposed CGWO.

When choosing the compared algorithms, besides the original GWO, the outstanding GWO improved versions are considered first. The IGWO(M) algorithm is regarded as one of the most promising and highly cited improved versions of GWO; it shows very competitive optimization ability in solving engineering problems over the well-regarded algorithms such as Whale Optimization Algorithm (WOA), Exploration-Enhanced Grey Wolf Optimizer (EEGWO), and Henry Gas Solubility Optimization (HGSO). Moreover, the IGWO(A) and ExGWO algorithms also belong to those highly referenced GWO variants and have been selected as the comparison algorithms by many researchers. Since two of the compared algorithms have the same abbreviations of IGWO, we distinguish them using the first letter of the first author's first name in the parentheses after the algorithms' names.

Then, those algorithms designed for training MLP are taken into account. AGWO is an improved version of GWO designed to solve MLP optimization problems. It has been verified as a superior tool to train MLP classifiers compared with some well-known and state-of-the-art algorithms, including PSO, GA, Biogeography-based Optimizer (BBO) and Butterfly Optimization Algorithm (BOA). RSMGWO is another GWO variant used to train MLPs and shows good performances over many other algorithms, including BBO, Ant Colony Optimization (ACO), Population-based Incremental Learning (PBIL), etc.

Further, the three other algorithms, BRO, MSCA, and RDE-OP, are also adopted for comparison in the experiment because these algorithms show excellent performance in the optimization for MLP training.

4.1.3. Methodology

The k-fold cross-validation is employed for evaluation, in which the dataset is randomly partitioned into k folds; one-fold is used to test the network, whereas the remaining k-1 folds are used as the training sets. Each of the k folds is selected once for the testing, and the process of training and testing is repeated k times. In this study, k is set to 5, and thus 5-fold cross-validation (5CV) is performed.

As the stochastic optimization techniques, the competing algorithms in the simulation experiment can lead to different results in different independent runs. To generate meaningful statistical results, all the empirical results are achieved by taking the average of 10 independent runs for each dataset. In each run, the average result of the 5-fold cross-validation is recorded.

For a fair comparison, all competing algorithms were implemented in MATLAB 2015a and performed on a personal computer with a 3.40 GHz CPU and 8 GB RAM under the Windows 7 operating system.

Moreover, all input features were mapped to the same target interval of $[max', min']$ to eliminate the effect of features that have different scales by using the min–max function given in Eq. (19) as follows:

$$x' = \frac{(x - min)\ \ (max' - min')}{(max - min)} + min' \tag{19}$$

where x' is the normalized value of x in the range $[min,\ \ max]$.

To evaluate the proposed CGWO technique, we compare the results obtained by CGWO and other competing algorithms by quantifying the objective function value (fitness value) and classification accuracy. The objective function value, being the metric that most intuitively reflects the algorithm's optimization capability, is thus employed as the primary performance metric for evaluating experimental results. In addition, classification accuracy is also important for analyzing the results of MLP training. But considering that classification accuracy can mislead the results when data classes are imbalanced, it is used as an auxiliary metric to evaluate the algorithm's performance from another perspective. The classification accuracy includes two items: classification accuracy with training samples (training classification accuracy) and classification accuracy with test samples (test classification accuracy). The training and the test classification accuracies refer to the correct rate of classification decisions in the training and test sets, respectively. Moreover, to illustrate the computation efficiency of the competing algorithms, the CPU computation time and the convergence speed are investigated.

To be fair, the population size and the maximum number of objective function evaluations (FEs) of all competing algorithms for all datasets are set to 200 and $4.0 * 10^4$, respectively.

To demonstrate the central tendency and statistical dispersion of the objective function value, classification accuracy, and CPU computation time, the median ($\widetilde{x}$) and interquartile range (IQR) of the metrics for each competing algorithm on each dataset were calculated based on the average results of the 5-fold cross-validation. Moreover, in order to ensure reliable and confidential comparisons, the statistical analysis described in Refs. Yang, Wu, and Min (2015) was employed. The overall framework of the statistical analysis is outlined in Fig. 6. Firstly, the distribution of the results is assessed by conducting the Kolmogorov–Smirnov test to determine whether they are following a normal (Gaussian) distribution or not. If the results do not adhere to a normal distribution, the nonparametric Kruskal–Wallis test is performed to compare the result of each algorithm. Conversely, if the results do follow a normal distribution, the homogeneity of variances is checked using the Levene test. The confidence of comparisons is verified by conducting a Welch test if the samples have different variances; otherwise, an ANOVA test is carried out to accomplish this task. The statistical analysis is performed at a confidence level of 95%.

4.1.4. Parameters setting

The parameters for the original GWO, IGWO(M), AGWO, RSMGWO, ExGWO, IGWO(A), BRO, MSCA, and RDE-OP were set according to the corresponding references, respectively. Following the parameter configuration in the literature (Mirjalili, 2015), a is linearly decreased from 2 to 0 for the original GWO. The improved GWO versions, including IGWO(M), AGWO, RSMGWO, and ExGWO, all use the same method to configure the key parameter a, whereas IGWO(A) adopts the nonlinear decrease method for configuring a, and the initial and the final value of a is also set to 2 and 0, respectively. For BRO, the initial value for $\triangle$ is set to $\frac{maxCicle}{round(log_{10}(maxCicle))}$ as reported in the literature (Agahian & Akan, 2022), in which $maxCicle$ is the maximum iteration numbers. For MSCA, the parameter configuration of $a = 2$, $b = 0.5$, and $c = 4$, as reported in the literature (Gupta et al., 2020), are adopted. For RDE-OP, the scaling factor, crossover probability, jumping rate, and clustering period are set to 0.5, 0.9, 0.3, and 10, respectively, as reported in the literature (Mousavirad et al., 2021).

For the proposed CGWO algorithm, no additional parameters needed to be manually set for the MLP training problem, and the

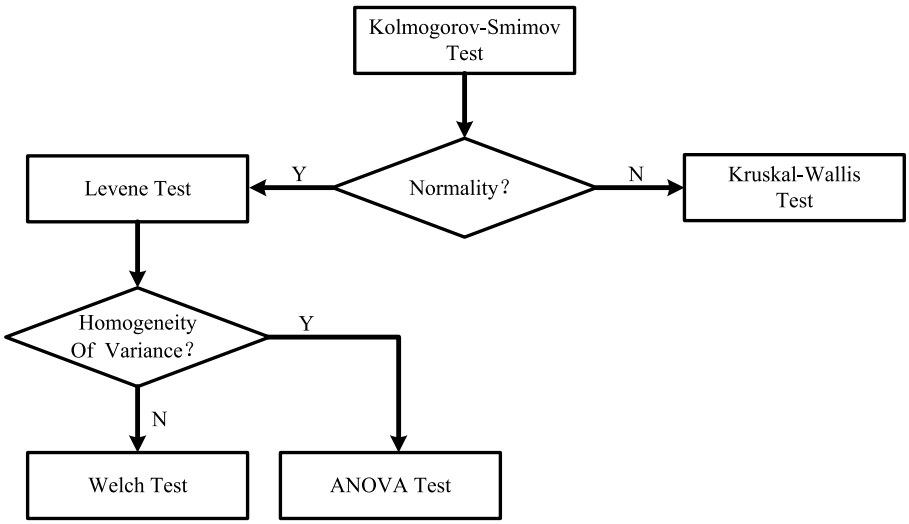


Fig. 6. The structure of the statistical analysis.

parameter configuration was the same as in the original GWO. And according to Eq. (13), M_{max} is automatically assigned different values based on the dimensionality of the problem. For the 11 datasets in this experiment, the corresponding values of M_{max} are 6, 11, 13, 22, 24, 25, 27, 40, 40, 40, and 40, respectively.

4.2. Experimental results

One of the most important issues for the SI optimization algorithm is efficiently searching for solutions with high accuracy and stability. Therefore, two main performance metrics, including the solution quality, and computation efficiency, of the proposed CGWO algorithm in comparison to the other algorithms, are analyzed in the first and second subsections.

4.2.1. Solution quality

Regarding the solution quality, firstly, Table 2 shows the numerical results of the competing algorithms in terms of the median and interquartile range (IQR) of the fitness (CEE) values in the 10 independent runs. The training and test classification accuracies achieved by the competing algorithms are shown in Tables 3 and 4, respectively.

For Table 2, a smaller median fitness value represents higher solution quality, and a lower interquartile range indicates better stability. While for Tables 3 and 4, a larger median fitness value represents higher classification accuracy, and a lower interquartile range indicates better stability.

In Tables 2 to 4, the best results for each dataset are highlighted with boldfaces. Moreover, to illustrate whether CGWO is significantly better than the other competitors, the statistical test was conducted following the flow illustrated in Fig. 6. And the statistical test results are presented in the last column of tables 2 to 4 by the symbols of "+ / ∗ / −", which indicate the result obtained by CGWO is superior to, equal to, and inferior to that of each of the corresponding compared algorithms, respectively. The first one of the symbols in each cell of the last column of the tables presents the statistically significant performance difference between CGWO and GWO. The rest symbols are the significant differences between CGWO and IGWO(M), AGWO, RSMGWO, ExGWO, IGWO(A), BRO, MSCA, and RDE-OP, in that order. In addition, to illustrate the performance differences between the competing algorithms in another manner, the box-and-whisker plots for the results of fitness values, training classification accuracy, and test classification accuracy are illustrated in Figs. 7 to 9, respectively.

In Tables 2, 3, and 4, among all the competing algorithms, the proposed CGWO obtains 11, 11, and 7 best median values with 8, 7, and 7 least IQR values on fitness value, training classification accuracy, and test classification accuracy, respectively. It can also be seen that, for those datasets where the CGWO is not ranked first, CGWO also has the top rankings in both the median and IQR values. That is evidence of the improvements of the CGWO in obtaining high-accuracy solutions with superior stability compared to the other optimizers.

The statistical test results also indicate the overwhelming performance of the CGWO over the other algorithms in the one-to-one comparisons since the total numbers of "+" of the statistical test results in Tables 2, 3, and 4 are 93, 86, and 77 out of 99, respectively. On the other hand, the numbers of "−" in Tables 2, 3, and 4 are only 0, 3, and 6 out of 99, respectively, which also illustrates that the CGWO is generally significantly better than the other algorithms. Further, it can be inferred from the results in Table 2 that the CGWO has an outstanding search capability in larger scale problems due to the results obtained by the CGWO are significantly better than the other competitors in those datasets with higher variable dimensions, especially in 4th (Statlog), 9th (Parkinson), 10th (Ionosphere), and 11th (Chess) datasets. The experimental results of fitness values directly verify that CGWO can avoid the local optima convergence and has a superior global search ability and search efficiency owing to the newly introduced mechanism of competing leaders.

Moreover, the box-and-whisker plots shown in Figs. 7 to 9 provide the visualization of the experimental results. Inspecting those plots, it is found that, the CGWO has the overall most efficient performance among all the competing algorithms in attaining the minimum fitness values, the maximum training classification accuracy, and the maximum test classification accuracy with mainly the best stability in consecutive runs.

It can be summarized from the results in Tables 2 to 4 and Figs. 7 to 9 that the CGWO significantly outperforms all the other competing algorithms in terms of fitness value, training classification accuracy, and test classification accuracy. Therefore, the CGWO can be regarded as a powerful tool for training MLPs whether the target dataset has a smaller number or a larger number of features and whether those datasets are involved in binary or more classifications.

4.2.2. Computational efficiency

The salient yardsticks to evaluate the computation efficiency of an optimization algorithm includes the computation time needed to generate the final solution and the convergence speed through the whole iteration.

Table 2
CEE.

Dataset	CGWO $\tilde{x}$(IQR)	GWO $\tilde{x}$(IQR)	IGWO(M) $\tilde{x}$(IQR)	AGWO $\tilde{x}$(IQR)	RSMGWO $\tilde{x}$(IQR)	ExGWO $\tilde{x}$(IQR)	IGWO(A) $\tilde{x}$(IQR)	BRO $\tilde{x}$(IQR)	MSCA $\tilde{x}$(IQR)	RDE-OP $\tilde{x}$(IQR)	Statistical test
Scale	1.2290E−01 (1.5700E−02)	1.4720E−01 (1.7700E−02)	1.3500E−01 (1.4700E−02)	1.9380E−01 (2.0700E−02)	1.7430E−01 (4.9300E−02)	2.6620E−01 (5.2100E−02)	3.1910E−01 (2.6800E−02)	1.8450E−01 (1.1100E−02)	6.7450E−01 (8.0100E−02)	2.2550E−01 (2.2400E−02)	+/+/+/+/+/+/+/+/+
Seeds	8.3000E−03 (3.6000E−03)	1.2600E−02 (3.7000E−03)	1.3700E−02 (3.5000E−03)	2.0300E−02 (2.6000E−03)	1.8500E−02 (8.2000E−03)	4.7400E−02 (1.2700E−02)	9.4100E−02 (3.2600E−02)	5.4100E−02 (1.1000E−02)	1.2350E−01 (1.9500E−02)	1.0810E−01 (6.6000E−03)	+/+/+/+/+/+/+/+/+
Cancer	1.1200E−02 (2.4000E−03)	2.5100E−02 (2.1000E−03)	2.2700E−02 (2.8000E−03)	3.6400E−02 (3.6000E−03)	2.7100E−02 (1.2800E−02)	4.5800E−02 (4.8000E−03)	7.2600E−02 (5.9000E−03)	3.7700E−02 (4.2000E−03)	7.8200E−02 (3.3000E−03)	9.4500E−02 (2.8000E−03)	+/+/+/+/+/+/+/+/+
Statlog	6.9000E−02 (1.9300E−02)	1.5220E−01 (1.3100E−02)	1.4470E−01 (1.7700E−02)	1.9920E−01 (1.4200E−02)	2.1830E−01 (6.1100E−02)	2.1600E−01 (1.4500E−02)	3.4780E−01 (3.9900E−02)	1.8350E−01 (1.2800E−02)	3.4000E−01 (1.5900E−02)	3.3620E−01 (1.8800E−02)	+/+/+/+/+/+/+/+/+
Wine	0.0000E+00 (0.0000E+00)	0.0000E+00 (0.0000E+00)	0.0000E+00 (0.0000E+00)	0.0000E+00 (0.0000E+00)	1.0000E−04 (8.0000E−04)	1.0000E−03 (6.0000E−04)	2.2800E−02 (1.4800E−02)	7.0000E−04 (3.0000E−04)	3.3160E−01 (2.2690E−01)	9.0000E−04 (1.1000E−03)	*/*/*/+/+/+/+/+/+
Australian	2.1000E−01 (6.9000E−03)	2.5410E−01 (6.6000E−03)	2.4900E−01 (6.3000E−03)	2.7560E−01 (5.2000E−03)	2.7930E−01 (2.9800E−02)	2.8260E−01 (5.9000E−03)	3.3900E−01 (1.0800E−02)	2.6930E−01 (9.9000E−03)	3.3250E−01 (8.5000E−03)	3.2750E−01 (8.9000E−03)	+/+/+/+/+/+/+/+/+
Cleveland	4.8110E−01 (2.7200E−02)	6.8160E−01 (1.0800E−02)	7.3570E−01 (3.1400E−02)	7.7330E−01 (1.3100E−02)	7.1060E−01 (1.0900E−02)	1.0400E+00 (3.9200E−02)	1.5949E+00 (1.2600E−01)	9.4650E−01 (4.3000E−02)	1.5019E+00 (8.5500E−02)	8.0460E−01 (1.4000E−02)	+/+/+/+/+/+/+/+/+
Zoo	0.0000E+00 (0.0000E+00)	0.0000E+00 (0.0000E+00)	0.0000E+00 (0.0000E+00)	0.0000E+00 (0.0000E+00)	0.0000E+00 (1.4000E−03)	8.4000E−03 (5.0000E−03)	3.0460E−01 (1.3930E−01)	5.9000E−03 (3.0000E−03)	1.5857E+00 (3.5340E−01)	1.3500E−02 (1.0400E−02)	*/*/*/+/+/+/+/+/+
Parkinson	2.7400E−02 (4.4000E−03)	4.1200E−02 (1.0300E−02)	4.3100E−02 (5.3000E−03)	6.7300E−02 (2.1700E−02)	8.0500E−02 (2.9400E−02)	1.0420E−01 (2.6000E−02)	2.1370E−01 (2.6000E−02)	9.2400E−02 (1.1900E−02)	2.6700E−01 (3.3500E−02)	2.8080E−01 (1.0100E−02)	+/+/+/+/+/+/+/+/+
Ionosphere	1.1000E−03 (4.0000E−04)	3.3000E−03 (1.9000E−03)	3.4000E−03 (1.8000E−03)	1.1300E−02 (4.6000E−03)	3.6000E−03 (1.6000E−03)	3.6400E−02 (6.5000E−03)	2.0070E−01 (2.5400E−02)	3.0100E−02 (4.9000E−03)	2.6710E−01 (6.9800E−02)	1.2910E−01 (3.6700E−02)	+/+/+/+/+/+/+/+/+
Chess	2.5200E−02 (1.2000E−03)	5.8000E−02 (8.4000E−03)	1.0100E−01 (5.9000E−03)	7.9900E−02 (9.2000E−03)	1.3100E−01 (2.9400E−02)	2.7640E−01 (1.4400E−02)	5.3030E−01 (1.0680E−01)	3.4670E−01 (2.7200E−02)	3.6550E−01 (8.0600E−02)	1.9130E−01 (1.2700E−02)	+/+/+/+/+/+/+/+/+

Table 3
Training classification accuracy.

Dataset	CGWO $\tilde{x}$(IQR)	GWO $\tilde{x}$(IQR)	IGWO(M) $\tilde{x}$(IQR)	AGWO $\tilde{x}$(IQR)	RSMGWO $\tilde{x}$(IQR)	ExGWO $\tilde{x}$(IQR)	IGWO(A) $\tilde{x}$(IQR)	BRO $\tilde{x}$(IQR)	MSCA $\tilde{x}$(IQR)	RDE-OP $\tilde{x}$(IQR)	Statistical test
Scale	7.4680E+01 (1.2800E+00)	7.2820E+01 (3.3600E+00)	7.3880E+01 (5.6000E−01)	6.9720E+01 (2.4000E+00)	7.0220E+01 (6.0000E+00)	5.6200E+01 (6.8000E+00)	4.9920E+01 (3.0800E+00)	6.4520E+01 (2.3200E+00)	3.8940E+01 (8.7200E+00)	6.6600E+01 (6.0800E+00)	*/+/+/+/+/+/+/+/+
Seeds	9.5060E+01 (1.3095E+00)	9.4226E+01 (1.4286E+00)	9.3810E+01 (1.6667E+00)	9.1250E+01 (9.5240E−01)	9.2143E+01 (1.6667E+00)	8.5476E+01 (1.5476E+00)	8.0238E+01 (5.2381E+00)	8.3274E+01 (1.4286E+00)	7.9345E+01 (1.6667E+00)	8.0952E+01 (4.6429E+00)	+/+/+/+/+/+/+/+/+
Cancer	9.5780E+01 (6.0830E−01)	9.3813E+01 (7.1460E−01)	9.3938E+01 (5.3590E−01)	9.2007E+01 (6.0840E−01)	9.3222E+01 (9.6650E−01)	9.0433E+01 (1.0735E+00)	8.9109E+01 (9.2930E−01)	9.1094E+01 (7.8600E−01)	8.8091E+01 (2.5758E+00)	8.8019E+01 (1.5045E+00)	+/+/+/+/+/+/+/+/+
Statlog	7.5417E+01 (2.7778E+00)	6.3565E+01 (1.7593E+00)	6.1852E+01 (1.1111E+00)	5.7037E+01 (3.5185E+00)	5.3796E+01 (9.6296E+00)	5.2870E+01 (4.7222E+00)	4.3611E+01 (7.4074E+00)	5.5185E+01 (2.9630E+00)	3.5694E+01 (7.3148E+00)	3.4213E+01 (3.2407E+00)	+/+/+/+/+/+/+/+/+
Wine	1.0000E+02 (0.0000E+00)	1.0000E+02 (0.0000E+00)	1.0000E+02 (0.0000E+00)	1.0000E+02 (0.0000E+00)	1.0000E+02 (2.7970E−01)	1.0000E+02 (1.4080E−01)	9.1356E+01 (2.3825E+00)	1.0000E+02 (1.4080E−01)	7.5878E+01 (9.7213E+00)	9.6629E+01 (1.1110E+00)	*/*/*/−/−/+/−/+/+
Australian	5.4891E+01 (1.4130E+00)	4.8080E+01 (1.3043E+00)	4.8533E+01 (1.8841E+00)	4.4094E+01 (1.6667E+00)	4.2319E+01 (4.7464E+00)	4.2391E+01 (1.0507E+00)	3.8841E+01 (2.7899E+00)	5.0978E+01 (1.8116E+00)	3.1033E+01 (2.2464E+00)	3.3460E+01 (3.8406E+00)	+/+/+/+/+/+/+/+/+
Cleveland	8.1777E+01 (1.5162E+00)	7.2097E+01 (1.3442E+00)	7.1715E+01 (2.0186E+00)	6.8140E+01 (1.6023E+00)	7.0879E+01 (1.0942E+00)	6.4394E+01 (1.0896E+00)	5.6776E+01 (1.9246E+00)	6.7591E+01 (1.5140E+00)	2.3014E+01 (1.1803E+01)	6.3636E+01 (1.7718E+00)	+/+/+/+/+/+/+/+/+
Zoo	1.0000E+02 (0.0000E+00)	1.0000E+02 (0.0000E+00)	1.0000E+02 (0.0000E+00)	1.0000E+02 (0.0000E+00)	1.0000E+02 (0.0000E+00)	1.0000E+02 (0.0000E+00)	9.1964E+01 (3.4321E+00)	1.0000E+02 (0.0000E+00)	3.6295E+01 (1.8445E+01)	9.8761E+01 (2.5000E−01)	*/*/*/*/*/+/*/+/+
Parkinson	8.6410E+01 (2.4359E+00)	8.3718E+01 (3.4615E+00)	8.2115E+01 (1.7949E+00)	7.8077E+01 (3.3333E+00)	7.7500E+01 (4.7436E+00)	6.7949E+01 (3.5897E+00)	6.3526E+01 (3.8462E+00)	6.9167E+01 (2.9487E+00)	5.3782E+01 (3.9744E+00)	5.2244E+01 (6.1538E+00)	+/+/+/+/+/+/+/+/+
Ionosphere	9.9751E+01 (1.4290E−01)	9.9074E+01 (5.7040E−01)	9.8860E+01 (6.4110E−01)	9.6686E+01 (7.1480E−01)	9.8467E+01 (1.0686E+00)	8.8922E+01 (1.3554E+00)	8.1160E+01 (1.8477E+00)	8.9847E+01 (1.4217E+00)	7.2328E+01 (6.3521E+00)	7.8139E+01 (4.9316E+00)	+/+/+/+/+/+/+/+/+
Chess	9.1841E+01 (6.7290E−01)	8.6092E+01 (1.9870E+00)	8.0976E+01 (2.4020E+00)	8.2916E+01 (1.1889E+00)	7.4232E+01 (7.9549E+00)	6.6865E+01 (5.0603E+00)	5.3372E+01 (7.1330E+00)	6.7976E+01 (2.8234E+00)	4.7161E+01 (1.7074E+01)	5.7854E+01 (3.2843E+00)	+/+/+/+/+/+/+/+/+

Table 4
Test classification accuracy.

Dataset	CGWO $\tilde{x}$(IQR)	GWO $\tilde{x}$(IQR)	IGWO(M) $\tilde{x}$(IQR)	AGWO $\tilde{x}$(IQR)	RSMGWO $\tilde{x}$(IQR)	ExGWO $\tilde{x}$(IQR)	IGWO(A) $\tilde{x}$(IQR)	BRO $\tilde{x}$(IQR)	MSCA $\tilde{x}$(IQR)	RDE-OP $\tilde{x}$(IQR)	Statistical test
Scale	7.3920E+01 (2.0800E+00)	7.2880E+01 (2.7200E+00)	7.2960E+01 (8.0000E−01)	6.8160E+01 (4.3200E+00)	6.8560E+01 (4.1600E+00)	5.5040E+01 (8.3200E+00)	4.9920E+01 (4.0000E+00)	6.4160E+01 (3.3600E+00)	3.6720E+01 (1.0720E+01)	6.7600E+01 (4.1600E+00)	*/*/+/+/+/+/+/+/+
Seeds	8.9048E+01 (1.4286E+00)	8.7143E+01 (2.8571E+00)	8.8333E+01 (9.5240E−01)	8.7381E+01 (2.3810E+00)	8.7619E+01 (1.9048E+00)	8.2857E+01 (9.5240E−01)	7.8333E+01 (6.6667E+00)	8.1905E+01 (1.4286E+00)	7.7619E+01 (3.8095E+00)	8.1667E+01 (3.3333E+00)	*/*/+/+/+/+/+/+/+
Cancer	9.3275E+01 (9.9790E−01)	9.1346E+01 (8.5820E−01)	9.1701E+01 (1.2857E+00)	9.0630E+01 (1.1439E+00)	9.1274E+01 (7.1330E−01)	8.9271E+01 (1.0021E+00)	8.8559E+01 (1.2888E+00)	8.9414E+01 (1.7194E+00)	8.6844E+01 (2.2837E+00)	8.7905E+01 (1.0062E+00)	+/+/+/+/+/+/+/+/+
Statlog	6.2593E+01 (2.5926E+00)	5.4630E+01 (3.3333E+00)	5.4074E+01 (3.7037E+00)	5.1852E+01 (4.0741E+00)	4.9074E+01 (7.4074E+00)	4.5741E+01 (6.2963E+00)	4.0926E+01 (3.7037E+00)	5.1296E+01 (4.4444E+00)	3.6111E+01 (8.5185E+00)	3.6296E+01 (5.1852E+00)	+/+/+/+/+/+/+/+/+
Wine	9.4936E+01 (1.6508E+00)	9.5770E+01 (1.6984E+00)	9.5452E+01 (3.3968E+00)	9.6056E+01 (5.8730E−01)	9.3246E+01 (1.7143E+00)	8.9857E+01 (2.8095E+00)	8.2302E+01 (3.3810E+00)	8.7619E+01 (2.8571E+00)	6.8976E+01 (1.1127E+01)	9.6905E+01 (1.6508E+00)	−/−/−/+/+/+/+/+/−
Australian	5.3261E+01 (4.3478E+00)	4.7029E+01 (3.0435E+00)	4.5072E+01 (2.8986E+00)	4.2174E+01 (4.7826E+00)	4.2174E+01 (6.6667E+00)	4.0072E+01 (3.3333E+00)	4.0507E+01 (3.0435E+00)	4.9275E+01 (4.9275E+00)	3.1812E+01 (5.2174E+00)	3.4565E+01 (5.2174E+00)	+/+/+/+/+/+/+/+/+
Cleveland	5.3379E+01 (3.0508E+00)	5.4362E+01 (3.3898E+00)	5.1864E+01 (3.0169E+00)	5.4686E+01 (2.4011E+00)	5.3678E+01 (2.0000E+00)	5.2192E+01 (1.3616E+00)	5.0658E+01 (1.6836E+00)	4.9989E+01 (4.0226E+00)	2.2754E+01 (1.2525E+01)	6.2946E+01 (2.6836E+00)	*/*/*/*/*/+/+/+/−
Zoo	9.5114E+01 (1.0000E+00)	9.5090E+01 (1.0000E+00)	9.3588E+01 (2.7569E+00)	9.5090E+01 (2.0000E+00)	9.4186E+01 (3.8521E+00)	8.7728E+01 (6.0050E+00)	7.9816E+01 (2.6140E+00)	8.7723E+01 (2.1579E+00)	3.4873E+01 (1.9356E+01)	9.9048E+01 (1.9048E+00)	*/+/*/*/+/+/+/+/−
Parkinson	7.7692E+01 (6.6667E+00)	7.4872E+01 (3.0769E+00)	7.4359E+01 (3.0769E+00)	7.2820E+01 (2.5641E+00)	7.1538E+01 (7.1795E+00)	6.6410E+01 (3.5897E+00)	6.2564E+01 (7.1795E+00)	6.6154E+01 (3.5897E+00)	5.6923E+01 (8.7179E+00)	5.6667E+01 (2.5641E+00)	*/+/+/+/+/+/+/+/+
Ionosphere	8.6459E+01 (1.9920E+00)	8.7177E+01 (3.1308E+00)	8.7316E+01 (1.1469E+00)	8.4769E+01 (1.9718E+00)	8.7179E+01 (2.5634E+00)	8.0638E+01 (3.3883E+00)	7.6207E+01 (3.1469E+00)	7.9918E+01 (3.1268E+00)	7.0076E+01 (1.1698E+01)	7.7924E+01 (3.7183E+00)	*/*/+/*/+/+/+/+/+
Chess	9.0050E+01 (6.8900E−01)	8.5498E+01 (2.5356E+00)	7.9677E+01 (3.3462E+00)	8.2651E+01 (1.1565E+00)	7.3161E+01 (8.0734E+00)	6.5988E+01 (2.8833E+00)	5.2865E+01 (8.1014E+00)	6.6818E+01 (2.5057E+00)	4.7525E+01 (1.7279E+01)	5.7431E+01 (3.8533E+00)	+/+/+/+/+/+/+/+/+

4.2.2.1. Computation time. The computation time for each independent run of each competing algorithm on each dataset was recorded (in seconds), and the corresponding median and interquartile range of the computation time in 10 runs are summarized in Table 5. And the statistical test was also achieved to check the significant differences between the proposed CGWO and each of the compared algorithms. The corresponding results are also illustrated with "+ / ∗ / −", which have the same meanings as those symbols in Tables 2 to 4. Further, the box-and-whisker plots on computation time for all competing algorithms are illustrated in Fig. 10.

The statistical test results in Table 5 and the box-and-whisker plots in Fig. 10 illustrate that the computation speed of the CGWO ranks in the middle of all competing algorithms. Since extra computational effort is needed in the mechanism of competing leaders, in theory, the CGWO is slower than the original GWO to complete the MLPs training process. And the results in Tables 5 and Fig. 10 also provide evidence for this conclusion. However, it can be seen that the computation time needed by the CGWO is not much more than what the GWO needs. That is a natural result of the fact that, in the form of a big O-notation, CGWO has the same time complexity as GWO. Moreover, even compared with the fastest algorithms, the inferiority of the CGWO is not very significant. In most cases, the additional computation time of the CGWO is less than 10%, compared with the fastest algorithm on each dataset in the experiment individually. Therefore, the margin in computation speed between CGWO and the fastest algorithm in the experiment is inconsiderable.

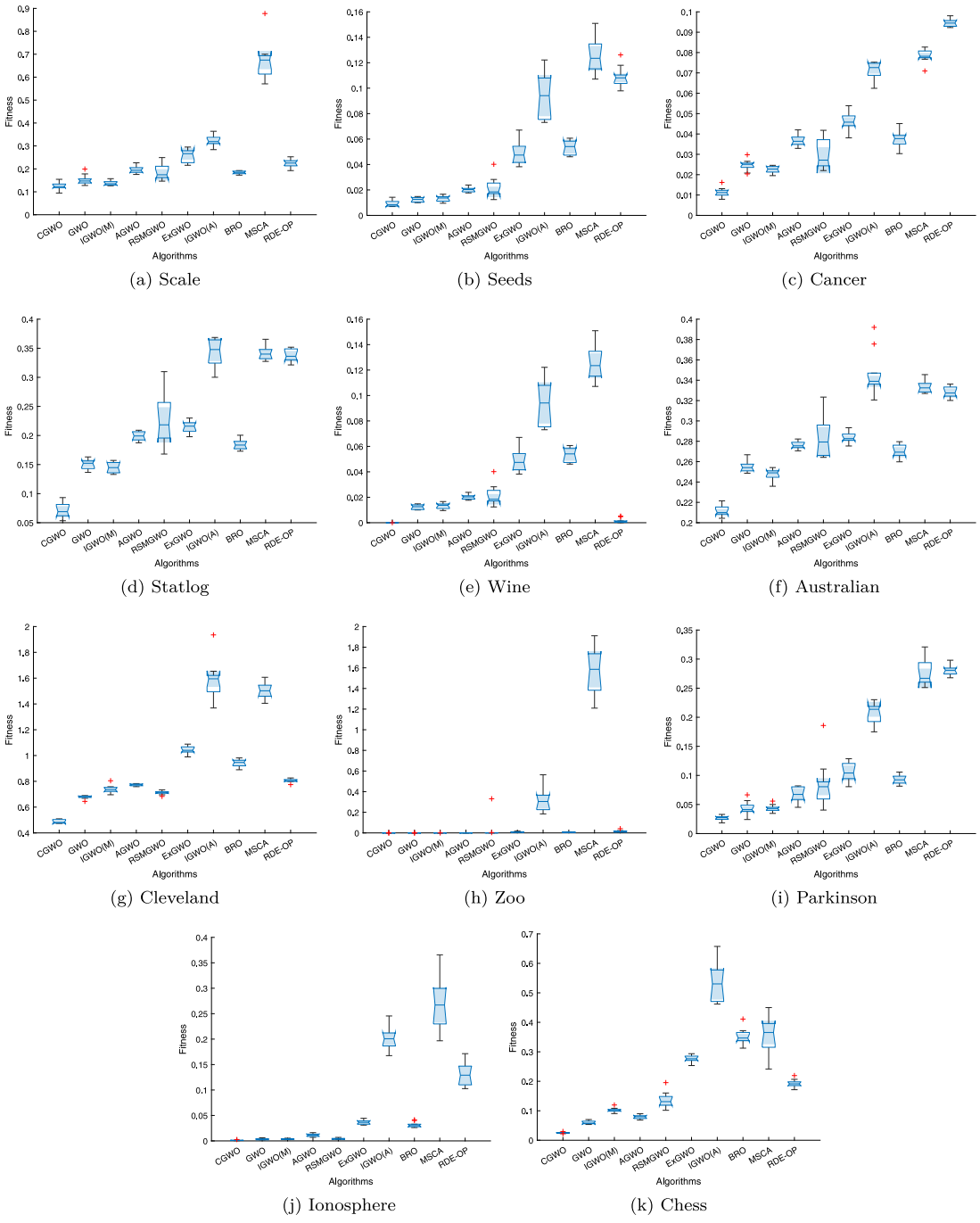


Fig. 7. Box-and-whisker plots of CEE.

4.2.2.2. Convergence speed. In order to do a deep investigation of the process of the competing algorithms in approximating their final optima in the experiment, the convergence speed analysis was performed for CGWO and its competitors, and thus the convergence curves are drawn.

For each convergence curve, the best objective function value obtained by the corresponding algorithm in every 400 additional function evaluations (FEs) was recorded as the data point. Moreover, since the convergence performance of a stochastic optimization algorithm cannot be judged by the result in a certain run, the median values of the data points over the 10 independent runs are used to illustrate the overall behaviors of the competing algorithms in the convergence process. Each data point of the convergence curve is calculated as follows:

$$cov_{md}(i) = MD\left(d_1(i), d_2(i), \ldots, d_{10}(i)\right) \tag{20}$$

where $d_j(i)$ indicates the ith data point in the jth run, MD is a function generating the median result.

Fig. 11 portrays the convergence processes of all ten competing algorithms in the eleven datasets. The convergence curve of CGWO plunges rapidly in all of the eleven datasets. And in seven of the datasets, including Scale, Seeds, Cancer, Statlog, Wine, Australian, and

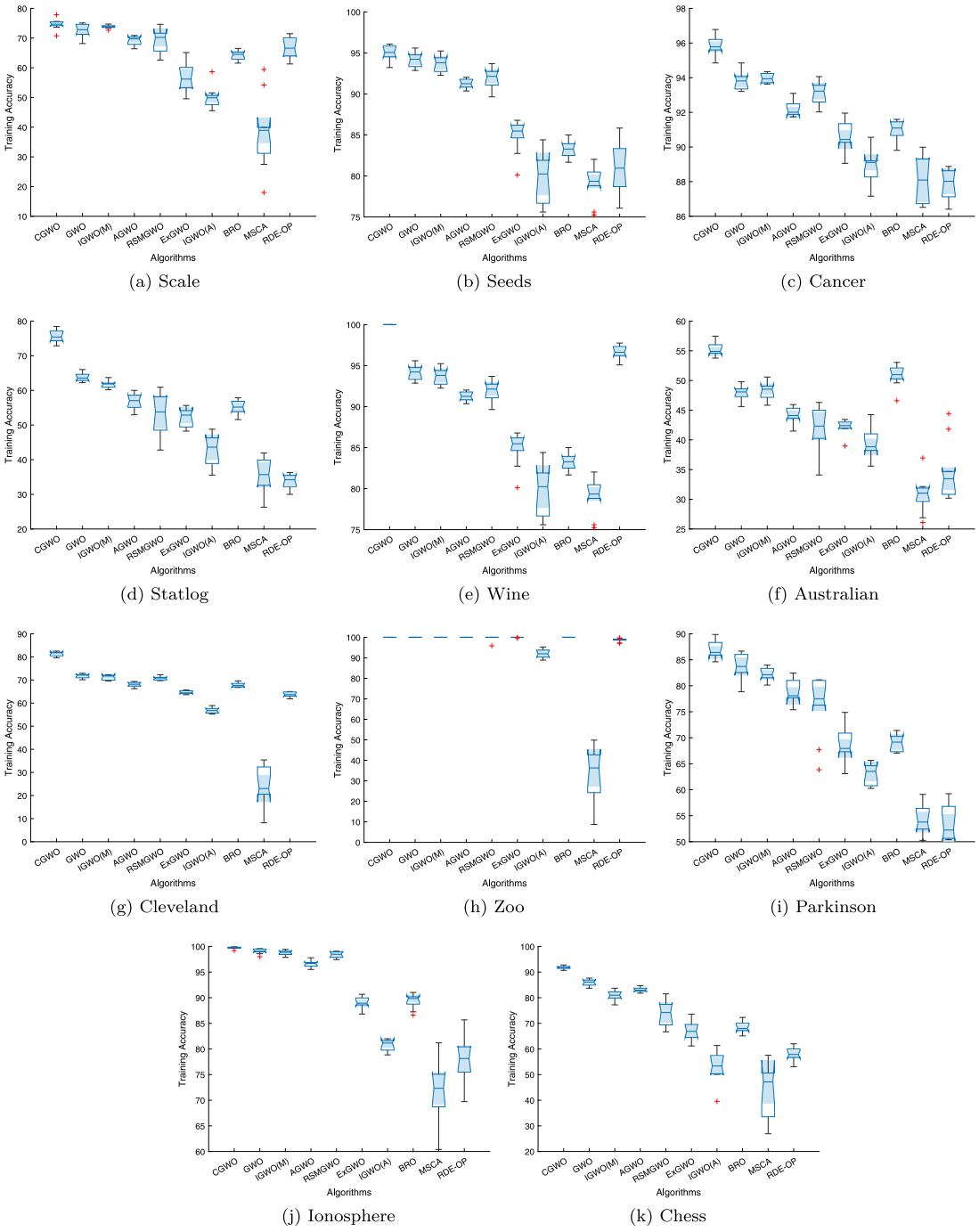


Fig. 8. Box-and-whisker plots of training classification accuracy.

Ionosphere, CGWO has the first leading convergence speed against all the nine competitors throughout the whole iteration. In the rest four datasets, including Cleveland, Zoo, Parkinson, and Chess, in the early stage of the iteration, CGWO ranks second and slightly falls behind MSCA or BRO, respectively. And in the later stage, CGWO surpasses the previous leading algorithms with a rapid descending trend in the four datasets. It is easy to understand that, on the larger scale problems, the population diversity enhancement strategy of CGWO is more likely to weaken the population's precise search capability in obtaining a better solution in the earlier stage. But after a short period, CGWO shows superior ability in approximating the final optimum faster since a more promising area in the search space is more likely to be found by the diversified search agents in the earlier iteration. Furthermore, the convergence curves of CGWO are generally smooth in the whole iteration process and almost plunge to the final solution without stagnation. In contrast, stagnations can be found in most of the convergence curves of the other competing algorithms. This implies the strong capability of CGWO to avoid falling into local optima.

All in all, Fig. 11 demonstrates that the CGWO has the best convergence speed and approximation accuracy, which can be attributed to

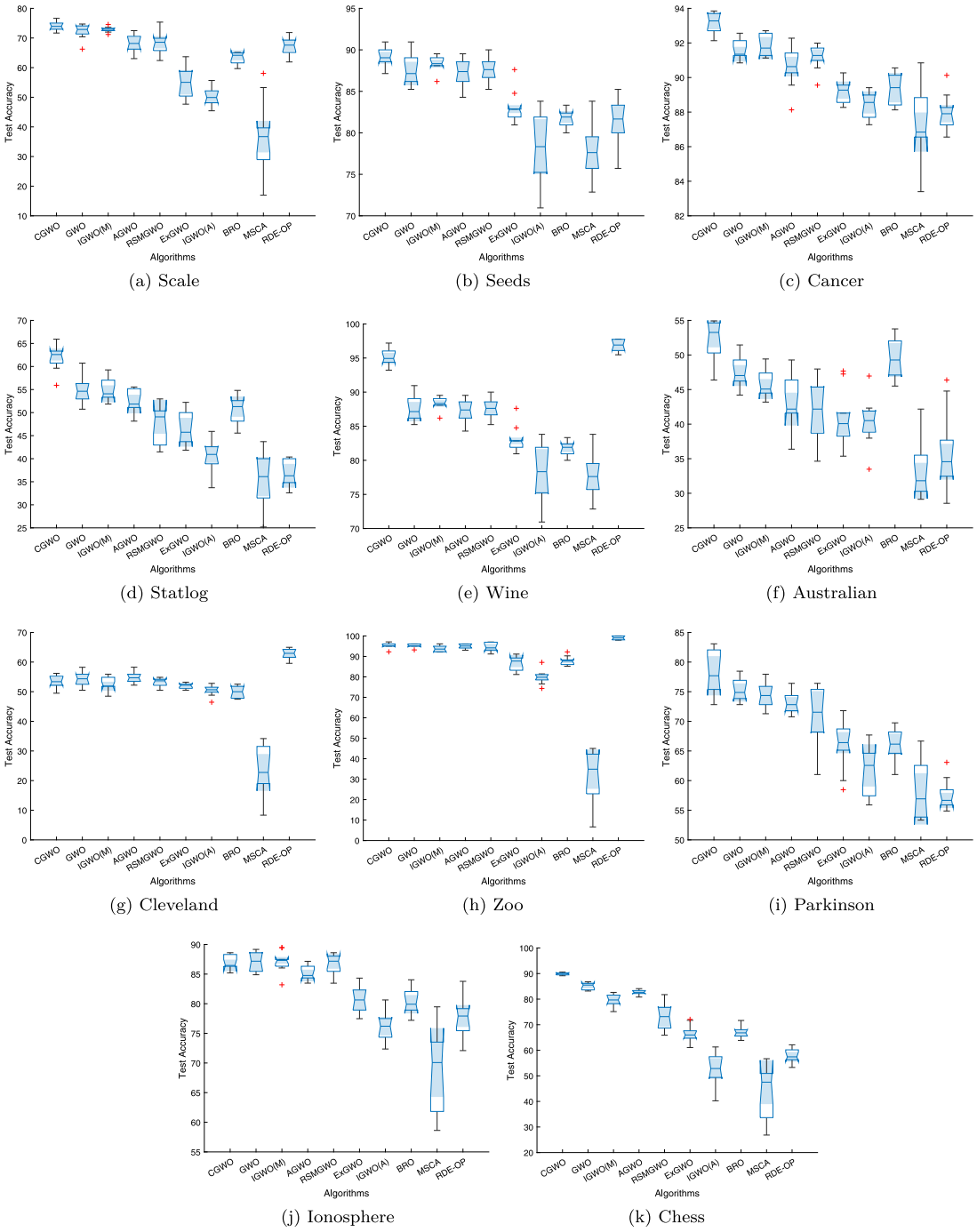


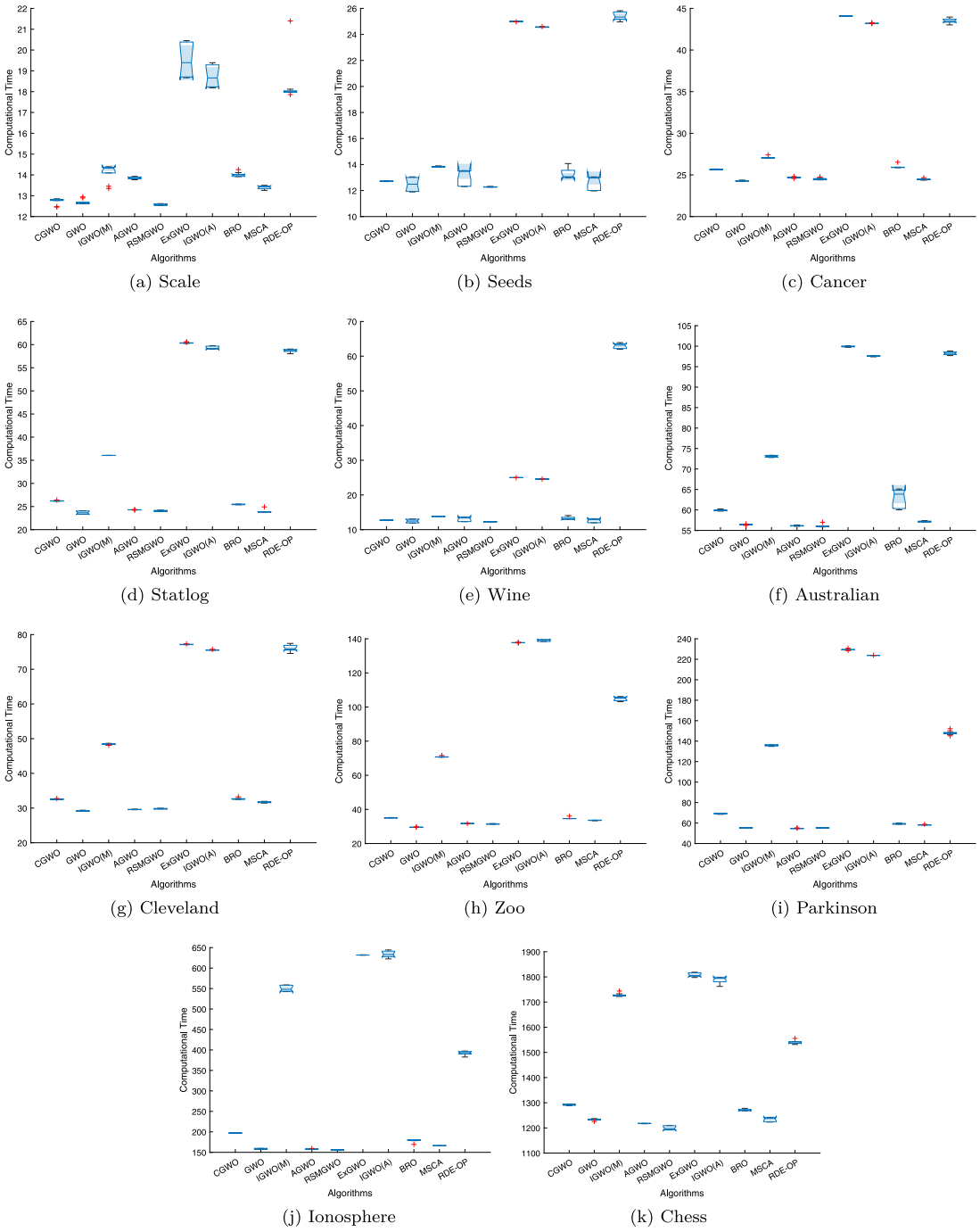
Fig. 9. Box-and-whisker plots of test classification accuracy.

its superior balance between exploration and exploitation throughout the whole iteration.

5. Conclusion

In this study, a novel competing leaders grey wolf optimizer (CGWO) is proposed and used for training the MLP classifier to get the optimal value of network parameters. Under comparison with nine other state-of-the-art algorithms, the proposed CGWO-MLP trainer is applied to eleven challenging UCI machine learning datasets that represent a wide range of optimization challenges and cover various fields, including healthcare, agriculture, communication, and finance. The experimental results confirm that CGWO substantially outperforms the original GWO and five outstanding GWO improved versions, IGWO(M), AGWO, RSMGWO, ExGWO, and IGWO(A), and the other three popular SI optimization algorithms, including BRO, MSCA, and RDE-OP.

The superiority of CGWO to approximate the global optima accurately, stably, and fastly over the other competing algorithms can be



(a) Scale (b) Seeds (c) Cancer

(d) Statlog (e) Wine (f) Australian

(g) Cleveland (h) Zoo (i) Parkinson

(j) Ionosphere (k) Chess

Fig. 10. Box-and-whisker plots of computation time.

attributed to the mechanism of competing leaders that can avoid stagnation in local optima and accelerate convergence speed. In addition, an initiation with population diversity enhancement also helps improve the search efficiency of the population. In terms of classification accuracies on both the training and test datasets, CGWO also yields promising results and outperforms all the competitors. Regarding the computation time to generate the final solution in the experiment, CGWO is in the middle of all algorithms and slower than the original GWO and some other competing algorithms with simpler implementations. This can be regarded as a drawback that limits the application of CGWO in scenarios where extremely fast processing is required. Nevertheless, the numerical comparison of the computation time illustrates that CGWO's time consumption in most cases is only slightly more than the fastest algorithms; thus, CGWO is still competitive for real-time applications. Therefore, it can be summed up that the proposed CGWO is an efficient and flexible optimization tool for training the MLP classifiers in various application fields.

The future work includes three aspects: (1) Exploring the efficiency of CGWO in accurately defining both the structure and connection weights and biases of MLPs would be an interesting area of investigation; (2) Regarding the further enhancement of CGWO, on the one hand, the CGWO can be improved following the ideas of a more delicate structure of grey wolves' leadership hierarchy; on the other hand, to improve the computational efficiency of CGWO, the implementation

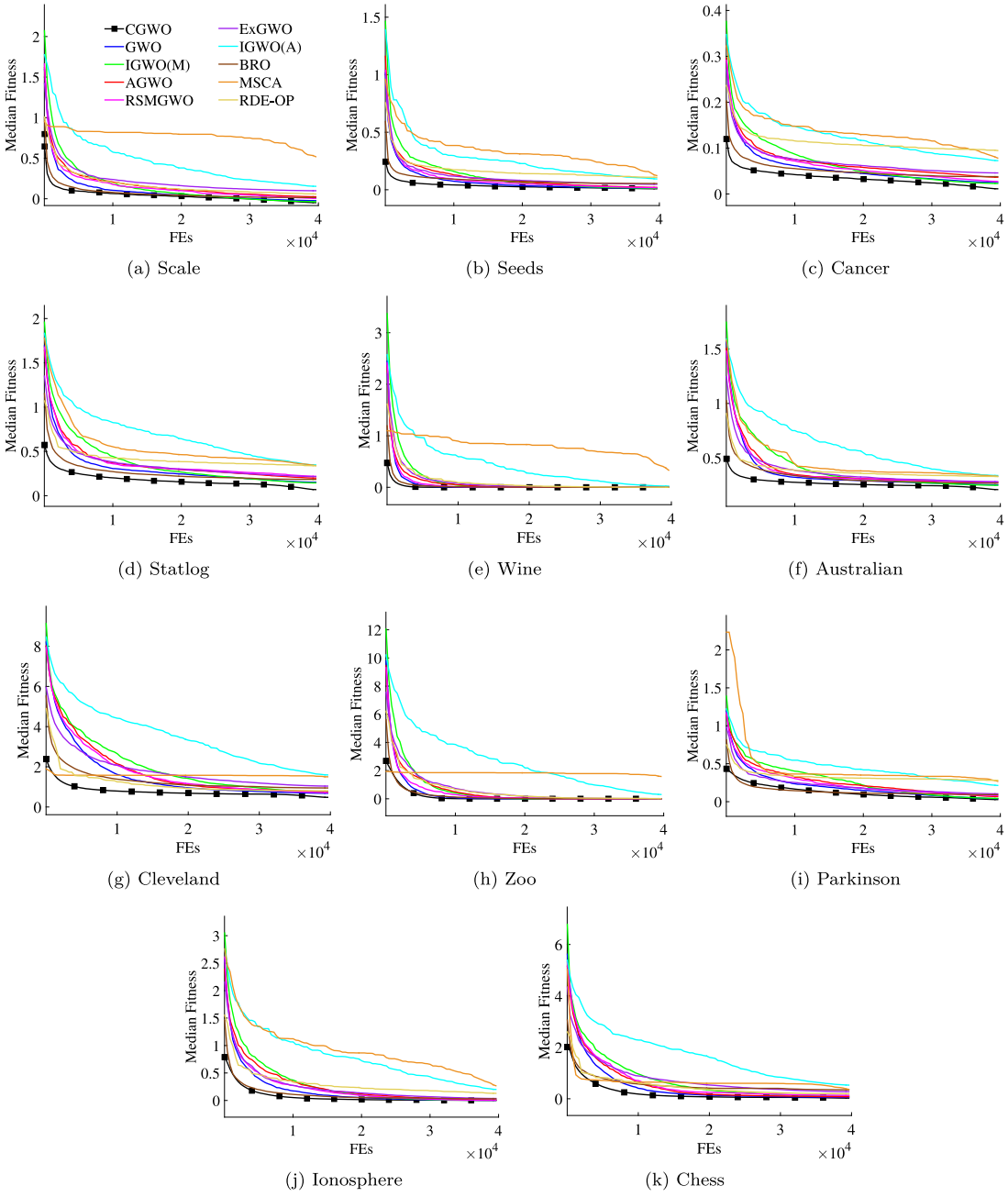


Fig. 11. Convergence curves of the competing algorithms.

of the mechanism of competing leaders with simpler methods can be investigated; (3) The CGWO also is expected to be used to solve other complex optimization problems in real-world applications.

CRediT authorship contribution statement

Zhenlun Yang: Conceptualization, Methodology, Software, Data curation, Visualization, Investigation, Validation, Supervision, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets from the UCI Machine Learning Repository (Aha et al., 2021) are used, which is available at https://archive.ics.uci.edu/datasets.

Acknowledgments

This study was funded by the Guangzhou Municipal Science and Technology Bureau of China (Research Grant no. 202002030133), the Department of Education of Guangdong Province of China (Research Grant no. 2022ZDZX1065), the Department of Science and Technology of Guangdong Province of China (Research Grant no. 2022A1515010999), the Guangzhou Municipal Education Bureau of China (Research Grant no. 201831785), the Guangzhou Panyu Polytechnic, China (Research Grant no. 2018KJ004).

Table 5
Computation time (measured in seconds).

Dataset	CGWO $\tilde{x}$(IQR)	GWO $\tilde{x}$(IQR)	IGWO(M) $\tilde{x}$(IQR)	AGWO $\tilde{x}$(IQR)	RSMGWO $\tilde{x}$(IQR)	ExGWO $\tilde{x}$(IQR)	IGWO(A) $\tilde{x}$(IQR)	BRO $\tilde{x}$(IQR)	MSCA $\tilde{x}$(IQR)	RDE-OP $\tilde{x}$(IQR)	Statistical test
Scale	1.2798E+01 (4.6800E−02)	1.2643E+01 (6.4100E−02)	1.4322E+01 (2.6280E−01)	1.3866E+01 (7.7200E−02)	1.2580E+01 (6.2200E−02)	1.9393E+01 (1.6792E+00)	1.8658E+01 (1.0564E+00)	1.4001E+01 (9.8400E−02)	1.3426E+01 (1.2610E−01)	1.7999E+01 (6.0300E−02)	*/+/+/−/+/+/+/+/+
Seeds	1.2711E+01 (2.3300E−02)	1.2487E+01 (1.0716E+00)	1.3820E+01 (4.8400E−02)	1.3474E+01 (1.1756E+00)	1.2274E+01 (2.5100E−02)	2.4991E+01 (1.5200E−02)	2.4569E+01 (2.2400E−02)	1.3065E+01 (5.4590E−01)	1.2959E+01 (1.0039E+00)	2.5334E+01 (5.6370E−01)	*/+/*/−/+/+/+/*/+
Cancer	2.5636E+01 (5.9600E−02)	2.4277E+01 (4.9200E−02)	2.7050E+01 (3.3900E−02)	2.4669E+01 (4.6200E−02)	2.4452E+01 (1.2090E−01)	4.4081E+01 (3.3900E−02)	4.3188E+01 (1.2800E−02)	2.5913E+01 (3.6100E−02)	2.4454E+01 (6.9500E−02)	4.3474E+01 (3.6330E−01)	−/+/−/−/+/+/+/−/+
Statlog	2.6212E+01 (4.6900E−02)	2.3640E+01 (6.9740E−01)	3.6050E+01 (1.2600E−02)	2.4285E+01 (2.1500E−02)	2.4005E+01 (2.1180E−01)	6.0344E+01 (5.5100E−02)	5.9132E+01 (5.7630E−01)	2.5449E+01 (1.0030E−01)	2.3838E+01 (4.5600E−02)	5.8784E+01 (3.9780E−01)	−/+/−/−/+/+/−/−/+
Wine	2.6940E+01 (1.5350E−01)	2.3309E+01 (4.3000E−02)	3.8714E+01 (1.6230E−01)	2.3864E+01 (6.8600E−02)	2.3535E+01 (6.8200E−02)	6.5980E+01 (1.0490E−01)	6.4809E+01 (2.1210E−01)	2.5402E+01 (2.4100E−02)	2.4107E+01 (7.9500E−02)	6.3200E+01 (1.3240E+00)	*/+/*/−/+/+/−/*/+
Australian	5.9868E+01 (2.0630E−01)	5.6394E+01 (8.1200E−02)	7.3093E+01 (4.1490E−01)	5.6107E+01 (9.9900E−02)	5.5996E+01 (7.7600E−02)	9.9939E+01 (1.4490E−01)	9.7574E+01 (1.2890E−01)	6.3862E+01 (4.4745E+00)	5.7140E+01 (2.0260E−01)	9.8257E+01 (6.1080E−01)	−/+/−/−/+/+/+/−/+
Cleveland	3.2574E+01 (9.4800E−02)	2.9094E+01 (1.3670E−01)	4.8423E+01 (1.5170E−01)	2.9557E+01 (8.0200E−02)	2.9766E+01 (1.3830E−01)	7.7135E+01 (8.0100E−02)	7.5545E+01 (7.8000E−02)	3.2609E+01 (1.4240E−01)	3.1648E+01 (2.4550E−01)	7.5866E+01 (1.3198E+00)	−/+/−/−/+/+/*/−/+
Zoo	3.5047E+01 (1.1660E−01)	2.9509E+01 (5.8100E−02)	7.0747E+01 (1.4250E−01)	3.1687E+01 (9.9000E−03)	3.1454E+01 (2.1800E−01)	1.3773E+02 (5.1700E−02)	1.3936E+02 (9.8300E−01)	3.4626E+01 (5.5700E−02)	3.3619E+01 (1.5700E−01)	1.0525E+02 (2.0193E+00)	−/+/−/−/+/+/−/−/+
Parkinson	6.9244E+01 (4.9650E−01)	5.5352E+01 (3.2170E−01)	1.3617E+02 (9.8910E−01)	5.4715E+01 (6.6500E−02)	5.5275E+01 (2.8060E−01)	2.2939E+02 (2.6990E−01)	2.2384E+02 (3.6200E−02)	5.9205E+01 (6.5670E−01)	5.8194E+01 (1.0550E−01)	1.4782E+02 (1.0964E+00)	−/+/−/−/+/+/−/−/+
Ionosphere	1.9690E+02 (7.7260E−01)	1.5851E+02 (2.4138E+00)	5.4808E+02 (1.5010E+01)	1.5782E+02 (3.6420E−01)	1.5585E+02 (3.0010E−01)	6.3190E+02 (2.5960E−01)	6.3316E+02 (1.3298E+01)	1.7987E+02 (1.6531E+00)	1.6615E+02 (3.2730E−01)	3.9330E+02 (5.6874E+00)	−/+/−/−/+/+/−/−/+
Chess	1.2922E+03 (4.5045E+00)	1.2331E+03 (2.8337E+00)	1.7265E+03 (3.6006E+00)	1.2176E+03 (8.9170E−01)	1.1967E+03 (1.5996E+01)	1.8051E+03 (1.2885E+01)	1.7949E+03 (1.5755E+01)	1.2701E+03 (4.5858E+00)	1.2378E+03 (1.5965E+01)	1.5377E+03 (6.4673E+00)	−/+/−/−/+/+/−/−/+

References

Adnan, R. M., Mostafa, R. R., Kisi, O., Yaseen, Z. M., Shahid, S., & Zounemat-Kermani, M. (2021). Improving streamflow prediction using a new hybrid ELM model combined with hybrid particle swarm optimization and grey wolf optimization. *Knowledge-Based Systems*, *230*, Article 107379.

Agahian, S., & Akan, T. (2022). Battle royale optimizer for training multi-layer perceptron. *Evolving Systems*, *13*(4), 563–575.

Aha, D., Murphy, P., Merz, C., Keogh, E., Blake, C., Hettich, S., et al. (2021). UCI machine learning repository. URL: http://archive.ics.uci.edu/ml/index.php.

AlShabi, M., Ghenai, C., Bettayeb, M., Ahmad, F. F., & El Haj Assad, M. (2021). Multi-group grey wolf optimizer (MG-GWO) for estimating photovoltaic solar cell model. *Journal of Thermal Analysis and Calorimetry*, *144*, 1655–1670.

Bacanin, N., Bezdan, T., Al-Turjman, F., & Rashid, T. A. (2022). Artificial flora optimization algorithm with genetically guided operators for feature selection and neural network training. *International Journal of Fuzzy Systems*, *24*(5), 2538–2559.

Bacanin, N., Budimirovic, N., K, V., Strumberger, I., Alrasheedi, A. F., & Abouhawwash, M. (2022). Novel chaotic oppositional fruit fly optimization algorithm for feature selection applied on COVID 19 patients' health prediction. *PLoS One*, *17*(10), Article e0275727.

Bacanin, N., Stoean, R., Zivkovic, M., Petrovic, A., Rashid, T. A., & Bezdan, T. (2021). Performance of a novel chaotic firefly algorithm with enhanced exploration for tackling global optimization problems: Application for dropout regularization. *Mathematics*, *9*(21), 2705.

Bajaj, A., Sangwan, O. P., & Abraham, A. (2022). Improved novel bat algorithm for test case prioritization and minimization. *Soft Computing*, *26*(22), 12393–12419.

Banaie-Dezfouli, M., Nadimi-Shahraki, M. H., & Beheshti, Z. (2021). R-GWO: Representative-based grey wolf optimizer for solving engineering problems. *Applied Soft Computing*, *106*, Article 107328.

Bansal, P., Kumar, S., Pasrija, S., & Singh, S. (2020). A hybrid grasshopper and new cat swarm optimization algorithm for feature selection and optimization of multi-layer perceptron. *Soft Computing*, *24*, 15463–15489.

Chakravarthy, V., Chowdary, P. S. R., Anguera, J., Mokara, D., & Satapathy, S. C. (2021). Pattern recovery in linear arrays using grasshopper optimization algorithm. In *Microelectronics, electromagnetics and telecommunications: proceedings of the fifth ICMEET 2019* (pp. 745–755). Springer.

Chakravarthy, V., Chowdary, P., Dib, N., & Anguera, J. (2022). Elliptical antenna array synthesis using evolutionary computing tools. *Arabian Journal for Science and Engineering*, *47*(3), 2807–2824.

Emary, E., Zawbaa, H. M., & Grosan, C. (2017). Experienced gray wolf optimization through reinforcement learning and neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, *29*(3), 681–694.

Emmanuel, D., Joseph, S., Oyewola, D., Fadele, A. A., Chiroma, H., et al. (2021). Application of grey wolf optimization algorithm: Recent trends, issues, and possible horizons. *Gazi University Journal of Science*, *35*(2), 485–504.

Fan, C., Zhou, Y., & Tang, Z. (2021). Neighborhood centroid opposite-based learning harris hawks optimization for training neural networks. *Evolutionary Intelligence*, *14*, 1847–1867.

Faris, H., Aljarah, I., Al-Betar, M. A., & Mirjalili, S. (2018). Grey wolf optimizer: a review of recent variants and applications. *Neural Computing and Applications*, *30*, 413–435.

Faris, H., Aljarah, I., & Mirjalili, S. (2016). Training feedforward neural networks using multi-verse optimizer for binary classification problems. *Applied Intelligence*, *45*, 322–332.

Gao, W.-f., & Liu, S.-y. (2012). A modified artificial bee colony algorithm. *Computers & Operations Research*, *39*(3), 687–697.

Guha, D., Roy, P. K., & Banerjee, S. (2016). Load frequency control of large scale power system using quasi-oppositional grey wolf optimization algorithm. *Engineering Science and Technology, an International Journal*, *19*(4), 1693–1713.

Guo, S., Wang, G., Han, L., Song, X., & Yang, W. (2022). COVID-19 CT image denoising algorithm based on adaptive threshold and optimized weighted median filter. *Biomedical Signal Processing and Control*, *75*, Article 103552.

Gupta, S., Deep, K., Mirjalili, S., & Kim, J. H. (2020). A modified sine cosine algorithm with novel transition parameter and mutation operator for global optimization. *Expert Systems with Applications*, *154*, Article 113395.

Gupta, S., Deep, K., Moayedi, H., Foong, L. K., & Assad, A. (2021). Sine cosine grey wolf optimizer to solve engineering design problems. *Engineering with Computers*, *37*, 3123–3149.

Jiang, Y., Tsai, P., Yeh, W.-C., & Cao, L. (2017). A honey-bee-mating based algorithm for multilevel image segmentation using Bayesian theorem. *Applied Soft Computing*, *52*, 1181–1190.

Kushwah, R., Kaushik, M., & Chugh, K. (2021). A modified whale optimization algorithm to overcome delayed convergence in artificial neural networks. *Soft Computing*, *25*(15), 10275–10286.

Li, Z. (2023). A local opposition-learning golden-sine grey wolf optimization algorithm for feature selection in data classification. *Applied Soft Computing*, *142*, Article 110319.

Li, Q., Chen, H., Huang, H., Zhao, X., Cai, Z., Tong, C., et al. (2017). An enhanced grey wolf optimization based feature selection wrapped kernel extreme learning machine for medical diagnosis. *Computational and Mathematical Methods in Medicine*, *2017*, 1–16.

Li, H., Li, J., Wu, P., You, Y., & Zeng, N. (2022). A ranking-system-based switching particle swarm optimizer with dynamic learning strategies. *Neurocomputing*, *494*, 356–367.

Lin, C., Cao, Z., & Zhou, M. (2022). Learning-based cuckoo search algorithm to schedule a flexible job shop with sequencing flexibility. *IEEE Transactions on Cybernetics*, *In press*, 1–13.

Lin, X., Yu, X., & Li, W. (2022). A heuristic whale optimization algorithm with niching strategy for global multi-dimensional engineering optimization. *Computers & Industrial Engineering*, *171*, Article 108361.

Mahdavi, S., Shiri, M. E., & Rahnamayan, S. (2015). Metaheuristics in large-scale global continues optimization: A survey. *Information Sciences*, *295*, 407–428.

Malakar, S., Ghosh, M., Bhowmik, S., Sarkar, R., & Nasipuri, M. (2020). A GA based hierarchical feature selection approach for handwritten word recognition. *Neural Computing and Applications*, *32*, 2533–2552.

Meidani, K., Hemmasian, A., Mirjalili, S., & Barati Farimani, A. (2022). Adaptive grey wolf optimizer. *Neural Computing and Applications*, *34*(10), 7711–7731.

Meng, X., Jiang, J., & Wang, H. (2021). AGWO: Advanced GWO in multi-layer perception optimization. *Expert Systems with Applications*, *173*, Article 114676.

Mirjalili, S. (2015). How effective is the grey wolf optimizer in training multi-layer perceptrons. *Applied Intelligence*, *43*, 150–161.

Mirjalili, S., & Gandomi, A. H. (2017). Chaotic gravitational constants for the gravitational search algorithm. *Applied Soft Computing*, *53*, 407–419.

Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014a). Grey wolf optimizer. *Advances in Engineering Software*, *69*, 46–61.

Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014b). Let a biogeography-based optimizer train your multi-layer perceptron. *Information Sciences*, *269*, 188–209.

Mousavirad, S. J., Schaefer, G., Korovin, I., & Oliva, D. (2021). RDE-OP: A region-based differential evolution algorithm incorporation opposition-based learning for optimising the learning process of multi-layer neural networks. In *Applications of evolutionary computation: 24th international conference, evoapplications 2021, held as part of evostar 2021, virtual event, april 7–9, 2021, proceedings 24* (pp. 407–420). Springer.

Nadimi-Shahraki, M. H., Taghian, S., & Mirjalili, S. (2021). An improved grey wolf optimizer for solving engineering problems. *Expert Systems with Applications*, *166*, Article 113917.

Nadimi-Shahraki, M. H., Taghian, S., Mirjalili, S., Zamani, H., & Bahreininejad, A. (2022). GGWO: Gaze cues learning-based grey wolf optimizer and its applications for solving engineering problems. *Journal of Computer Science*, *61*, Article 101636.

Rahkar Farshi, T. (2021). Battle royale optimization algorithm. *Neural Computing and Applications*, *33*(4), 1139–1157.

Rodríguez, A., Camarena, O., Cuevas, E., Aranguren, I., Valdivia-G, A., Morales-Castañeda, B., et al. (2021). Group-based synchronous-asynchronous grey wolf optimizer. *Applied Mathematical Modelling*, *93*, 226–243.

Safar, A. A., Salih, D. M., & Murshid, A. M. (2023). Pattern recognition using the multi-layer perceptron (MLP) for medical disease: A survey. *International Journal of Nonlinear Analysis and Applications*, *14*(1), 1989–1998.

Seyyedabbasi, A., & Kiani, F. (2021). I-GWO and ex-GWO: improved algorithms of the grey wolf optimizer to solve global optimization problems. *Engineering with Computers*, *37*(1), 509–532.

Singh, S., & Bansal, J. C. (2022). Mutation-driven grey wolf optimizer with modified search mechanism. *Expert Systems with Applications*, *194*, Article 116450.

Taib, H., & Bahreininejad, A. (2021). Data clustering using hybrid water cycle algorithm and a local pattern search method. *Advances in Engineering Software*, *153*, Article 102961.

Tan, L., Gong, D., Xu, J., Li, Z., & Liu, F. (2023). Meta-path fusion based neural recommendation in heterogeneous information networks. *Neurocomputing*, *529*, 236–248.

Teng, Z.-j., Lv, J.-l., & Guo, L.-w. (2019). An improved hybrid grey wolf optimization algorithm. *Soft Computing*, *23*, 6617–6631.

Wang, R., Hao, K., Chen, L., Wang, T., & Jiang, C. (2021). A novel hybrid particle swarm optimization using adaptive strategy. *Information Sciences*, *579*, 231–250.

Wang, B., Liu, L., Li, Y., & Khishe, M. (2022). Robust grey wolf optimizer for multimodal optimizations: A cross-dimensional coordination approach. *Journal of Scientific Computing*, *92*(3), 110.

Wang, Y., Zhu, Q., Ma, H., & Yu, H. (2022). A hybrid gray wolf optimizer for hyperspectral image band selection. *IEEE Transactions on Geoscience and Remote Sensing*, *60*, 1–13.

Yang, Z. (2023). FMFO: Floating flame moth-flame optimization algorithm for training multi-layer perceptron classifier. *Applied Intelligence*, *53*, 251–271.

Yang, Z., Shi, K., Wu, A., Qiu, M., & Wei, X. (2019). A hybird self-learning method based on particle swarm optimization and salp swarm algorithm. In *2019 tenth international conference on intelligent control and information processing (ICICIP)* (pp. 334–338). IEEE.

Yang, X., Wang, R., Zhao, D., Yu, F., Huang, C., Heidari, A. A., et al. (2023). An adaptive quadratic interpolation and rounding mechanism sine cosine algorithm with application to constrained engineering optimization problems. *Expert Systems with Applications*, *213*, Article 119041.

Yang, D., Wu, M., Li, D., Xu, Y., Zhou, X., & Yang, Z. (2022). Dynamic opposite learning enhanced dragonfly algorithm for solving large-scale flexible job shop scheduling problem. *Knowledge-Based Systems*, *238*, Article 107815.

Yang, Z.-L., Wu, A., & Min, H.-Q. (2015). An improved quantum-behaved particle swarm optimization algorithm with elitist breeding for unconstrained optimization. *Computational Intelligence and Neuroscience*, *2015*, 41.

Yavuz, G., Durmuş, B., & Aydın, D. (2022). Artificial bee colony algorithm with distant savants for constrained optimization. *Applied Soft Computing*, *116*, Article 108343.

Yu, X., Duan, Y., & Cai, Z. (2023). Sub-population improved grey wolf optimizer with Gaussian mutation and Lévy flight for parameters identification of photovoltaic models. *Expert Systems with Applications*, *232*, Article 120827.

Yu, L., Huo, S., Wang, Z., & Li, K. (2023). Hybrid attention-oriented experience replay for deep reinforcement learning and its application to a multi-robot cooperative hunting problem. *Neurocomputing*, *523*, 44–57.

Yu, X., & Wu, X. (2022). Ensemble grey wolf optimizer and its application for image segmentation. *Expert Systems with Applications*, *209*, Article 118267.

Yu, X., Xu, W., Wu, X., & Wang, X. (2022). Reinforced exploitation and exploration grey wolf optimizer for numerical and real-world optimization problems. *Applied Intelligence*, *52*, 8412–8427.

Zhang, X., Lin, Q., Mao, W., Liu, S., Dou, Z., & Liu, G. (2021). Hybrid particle swarm and grey wolf optimizer and its application to clustering optimization. *Applied Soft Computing*, *101*, Article 107061.

Zhang, X., Wang, X., Chen, H., Wang, D., & Fu, Z. (2020). Improved GWO for large-scale function optimization and MLP optimization in cancer identification. *Neural Computing and Applications*, *32*, 1305–1325.

Zhao, R., Wang, Y., Hu, P., Jelodar, H., Yuan, C., Li, Y., et al. (2019). Selfish herds optimization algorithm with orthogonal design and information update for training multi-layer perceptron neural network. *Applied Intelligence*, *49*, 2339–2381.

Zhu, L., Zhou, Y., Sun, S., & Su, Q. (2022). A discrete squirrel search algorithm for the surgical cases assignment problem. *Applied Soft Computing*, *121*, Article 108753.